数据结构与算法

授课教师：巢择敏

HIT 2026, Mar-Jun

摘要

这些笔记是对算法课程讲座的抄录记录，该课程于 2024 年 8 月至 11 月学期期间在 TIFR 举办，授课教师为乌南格・巴斯卡尔教授。课程的官方网页可通过此链接获取。

本课程的主要参考资料为 Cormen、Leiserson、Rivest 和 Stein (CLRS) 所著的《算法导论》，此外还辅以补充资料及授课教师本人的笔记，这些资料均可通过课程网页获取。对于关于近似算法的章节，还使用了 Vijay Vazirani 所著的《近似算法》中的第 14、15 和 17 章。

这些笔记既被用作个人的学术参考资料，也旨在帮助其他学生进行复习。它们已采用由我自行开发的定制主题进行排版。笔记的源代码可在此存储库中获取，而主题的源代码则可在我的 GitHub 仓库中找到。任何更正、评论或建议都欢迎提出，请通过 soham.chatterjee@tifr.res.in 联系我。

目录

Chapter 1 Finding Closest Pair of Points — Page 5

Chapter 2 线性表 — Page 6

2.1 线性表的定义和基本操作 6
2.1.1 线性表的定义 6
2.1.2 线性表的基本操作 7
2.2 线性表的顺序表示 7
2.2.1 顺序表的定义 7
2.2.2 顺序表上基本操作的实现 8
2.3 线性表的链式表示 10
2.3.1 单链表的定义 10
2.3.2 单链表上基本操作的实现 11
2.3.3 双链表 15
2.3.4 循环链表 16

Chapter 3 栈与队列 — Page 18

3.1 栈 18
3.1.1 栈的基本概念 18
3.1.2 栈的顺序存储结构 18
3.2 队列 21
3.2.1 队列的基本概念 21
3.2.2 队列的顺序存储结构 22
3.2.3 队列的链式存储结构 24
3.3 栈和队列的应用 26
3.3.1 栈在括号匹配中的应用 26
3.3.2 栈在表达式求值中的应用 27

Chapter 4 串 — Page 28

4.1 串的定义和实现 28
4.1.1 串的定义 28
4.1.2 串的基本操作 28

4.1.3 串的存储结构 29
4.2 串的模式匹配 30
4.2.1 简单的模式匹配算法 30
4.2.2 串的模式匹配算法——KMP 算法 30

CHAPTER 5 树与二叉树 PAGE 32
5.1 树的基本概念 32
5.1.1 树的定义 32
5.1.2 树的性质 32
5.2 二叉树的概念 33
5.2.1 二叉树的定义及其主要特性 33
5.2.2 二叉树的存储结构 34
5.3 二叉树的遍历和线索二叉树 35
5.3.1 二叉树的遍历 35
5.3.2 线索二叉树 36
5.4 树、森林 39
5.4.1 树的存储结构 39
5.4.2 树、森林与二叉树的转换 40
5.4.3 树和森林的遍历 41
5.5 树与二叉树的应用 42
5.5.1 哈夫曼树和哈夫曼编码 42
5.5.2 并查集 43

CHAPTER 6 图 PAGE 47
6.1 图的基本概念 47
6.1.1 图的定义 47
6.2 图的存储及基本操作 47
6.2.1 邻接矩阵法 47
6.2.2 邻接表法 48
6.2.3 十字链表法 49
6.2.4 邻接多重表法 49
6.2.5 图的基本操作 50
6.3 图的遍历 51
6.3.1 广度优先搜索（BFS） 51
6.3.2 深度优先搜索（DFS） 53
6.4 图的应用 54
6.4.1 最小生成树 54
6.4.2 最短路径 55
6.4.3 有向无环图描述表达式 57
6.4.4 拓扑排序 57
6.4.5 关键路径 58

Chapter 7 Kruskal's Algorithm with Data Structures — Page 59

Chapter 8 Red Black Tree Data Structure — Page 60

Chapter 9 Maximum Flow — Page 61

Chapter 10 Randomized Algorithm — Page 62

Chapter 11 Derandomization — Page 63

Chapter 12 Global Min Cut — Page 64

Chapter 13 Matching — Page 65

Chapter 14 Linear Programming — Page 66

Chapter 15 Approximation Algorithms using LP — Page 67

Chapter 16 P, NP and Reductions — Page 68

Chapter 17 参考文献 — Page 69

Chapter 1

Finding Closest Pair of Points

Chapter 2

线性表

线性表是一种常用的数据结构，本章要求掌握线性表的逻辑结构、顺序存储结构和链式存储结构的描述方法，熟练掌握线性表在顺序存储结构和链式存储结构中的特点以及相关的查找、插入、删除等基本操作的实现方法，并且能够从时间和空间复杂性的角度综合比较两种存储结构的不同特点。

2.1 线性表的定义和基本操作

2.1.1 线性表的定义

Definition 2.1.1: 线性表

线性表是由 $n(n \geq 0)$ 个相同数据类型的元素组成的有限序列。其中 n 为表长；当 $n = 0$ 时，线性表为空表。若用 L 表示一个线性表，则其一般形式为

$$L = (a_1, a_2, \cdots, a_i, a_{i+1}, \cdots, a_n)$$

式中，a_1 是唯一的“第一个”数据元素，也称表头元素；a_n 是唯一的“最后一个”数据元素，也称表尾元素。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继（“直接前驱”与“前驱”、“直接后继”与“后继”通常被视为同义词）。

由此，可总结出线性表的特点如下：

- 表中的元素的个数有限。
- 表中元素具有逻辑上的顺序性，即存在明确的先后次序。
- 表中元素均为数据元素，每个元素在线性表中被视为一个不可分割的单位。
- 表中元素的数据类型都相同，这意味着每个元素占有相同大小的存储空间。
- 表中元素具有抽象性，即仅讨论元素之间的逻辑关系，而不考虑其究竟表示什么内容。

Note:-

线性表是一种逻辑结构，表示元素之间一对一的相邻关系。顺序表和链表是两种存储结构，二者属于不同层面的概念，因此不要将其混淆。

2.1.2 线性表的基本操作

一个数据结构的基本操作是指其最核心、最基本的操作集合，其他较复杂的功能通常可通过调用这些基本操作来实现。线性表的主要操作如下：

- **InitList (&L)**：初始化表。构造一个空的线性表。
- **Length (L)**：求表长。返回线性表 L 的长度，即其中数据元素的个数。
- **LocateElem (L, e)**：按值查找。在表 L 中查找值为 e 的元素，并返回其位置。
- **GetElem (L, i)**：按位查找操作。获取表中第 i 个位置的元素值。
- **ListInsert (&L, i, e)**：插入操作。在表的第 i 个位置插入指定元素 e。
- **ListDelete (&L, i, &e)**：删除操作。删除表中第 i 个位置的元素，并用 e 返回被删元素的值。
- **PrintList (L)**：输出操作。按前后顺序输出线性表 L 的所有元素值。
- **Empty (L)**：判空操作。若 L 为空表，则返回 true，否则返回 false。
- **DestroyList (&L)**：销毁操作。销毁线性表，释放其占用的内存空间。

Note:-

① 基本操作的具体实现取决于所用的存储结构，存储结构不同，算法的实现方式也不同。

② 符号 & 表示 C++ 语言中的引用调用；在 C 语言中，可通过指针实现相同效果。

2.2 线性表的顺序表示

2.2.1 顺序表的定义

Definition 2.2.1: 顺序表

线性表的顺序存储也称顺序表。它是用一组地址连续的存储单元依次存储线性表中的数据元素，使得逻辑上相邻的元素在物理位置上也相邻。第 i 个元素存储在顺序表的起始位置，第 $i+1$ 个元素的存储位置之后紧接着存放的是第 $i+1$ 个元素，称 i 为元素 a_i 在顺序表中的位序。因此，顺序表的特点是：表中元素的逻辑顺序与其物理存储顺序完全一致。

由于任意数据元素的存储地址与顺序表起始地址之间的偏移量与其位序呈线性关系，因此可在 $O(1)$ 时间内直接访问表中任意位置的元素，这种特性使得线性表的顺序存储结构属于随机存取。在高级程序设计语言中，通常使用数组来实现顺序表。

Note:-

线性表中元素的位序从 1 开始，而数组中元素的下标从 0 开始，注意二者之间的转换。

假定线性表的元素类型为 ElemType，则静态分配和动态分配的顺序表存储结构描述为

Algorithm 1: 静态分配的顺序表存储结构

```
Input: 无
Output: 顺序表类型 SqList
Define SqList Structure:
    #define MaxSize 50
    typedef struct {
        ElementType data[MaxSize]
        int length
    } SqList
```

Algorithm 2: 动态分配的顺序表存储结构

```
Input: 无
Output: 顺序表类型 SeqList
Define SeqList Structure:
    #define InitSize 100
    typedef struct {
        ElementType *data
        int MaxSize, length
    } SeqList
```

一维数组既可以静态分配，也可以动态分配。静态分配时，数组的大小和存储空间在编译时已经固定，一旦空间占满，再插入新元素将导致溢出，进而可能引发程序异常。

动态分配时，存储数组的空间是在程序执行过程中通过动态存储分配语句申请的。当空间占满时，可以另行开辟一块更大的存储空间，将原表中的所有元素复制到新空间中，从而实现存储容量的扩充，而无须在初始化时为线性表一次性划分全部可能用到的空间。

Note:-

动态分配并不是链式存储，它同样属于顺序存储结构。其物理结构没有变化，依然支持随机存取, 只是存储空间的大小可以在运行时动态调整。

顺序表的主要优点：① **可进行随机访问**，即通过首地址和元素序号可在 $O(1)$ 时间内找到指定的元素；② **存储密度高**，每个结点仅存储数据元素，无额外指针开销。顺序表的缺点也很明显：① 插入和删除操作效率较低，需要移动大量元素；② 要求分配连续的存储空间，不够灵活。

2.2.2 顺序表上基本操作的实现

Note:-

在各种操作的实现中，通常可以忽略边界条件判断、变量定义、内存分配失败等实现细节，即不要求代码具有实际可执行性，而应重点体现算法的核心思想与逻辑步骤。

插入操作

在顺序表 L 的第 i $(1 \le i \le L.\text{length}+1)$ 个位置插入新元素 e。若 i 超出合法范围，或存储空间已满，则插入失败，返回 **false**；否则，将第 i 个元素及其后的所有元素依次后移一位，腾出一个空位置插入 e，表长加 1，返回 **true**。

Algorithm 3: ListInsert(L, i, e)

```
Input: 顺序表 L，插入位置 i（从 1 开始），待插入元素 e
Output: 插入成功返回 true，失败返回 false
begin
    if i < 1 or i > L.length + 1 then
        return false
    if L.length ≥ MaxSize then
        return false
    for j = L.length; j ≥ i; j − − do
        L.data[j] ⟵ L.data[j − 1]
    L.data[i − 1] ⟵ e
    L.length ⟵ L.length + 1
    return true
```

Note:-

区分位序（从 1 开始）与数组下标（从 0 开始）。为什么判断插入位置时使用 length+1，而移动元素的 for 循环中使用 length？因为合法插入位置包括第 $n+1$ 位，但移动元素时最多从第 n 位开始后移。

最好情况： 在表尾插入 $(i = n+1)$，无须移动元素，时间复杂度为 $O(1)$。

最坏情况： 在表头插入 $(i = 1)$，需移动全部 n 个元素，时间复杂度为 $O(n)$。

平均情况： 设在第 i 个位置插入的概率为 $p_i = 1/(n+1)$，则平均移动次数为

$$\sum_{i=1}^{n+1} p_i(n-i+1) = \sum_{i=1}^{n+1} \frac{1}{n+1}(n-i+1) = \frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1) = \frac{1}{n+1}\cdot\frac{n(n+1)}{2} = \frac{n}{2}$$

因此，插入操作的平均时间复杂度为 $O(n)$。

删除操作

删除顺序表 L 中第 i $(1 \le i \le L.\text{length})$ 个位置的元素，并通过引用参数 e 返回其值。若 i 非法，返回 **false**；否则，保存被删元素，将其后所有元素前移一位，表长减 1，返回 **true**。

Algorithm 4: LISTDELETE(L, i, e)

Input: 顺序表 L，删除位置 i（从 1 开始），用于返回被删元素的变量 e
Output: 删除成功返回 **true**，失败返回 **false**

```
begin
    if i < 1 or i > L.length then
        return false
    e ⟵ L.data[i − 1]
    for j = i; j < L.length; j++ do
        L.data[j − 1] ⟵ L.data[j]
    L.length ⟵ L.length − 1
    return true
```

最好情况： 删除表尾元素 $(i = n)$，无须移动元素，时间复杂度为 $O(1)$。

最坏情况： 删除表头元素 $(i = 1)$，需移动其余 $n-1$ 个元素，时间复杂度为 $O(n)$。

平均情况： 设删除第 i 个元素的概率为 $p_i = 1/n$，则平均移动次数为

$$\sum_{i=1}^{n} p_i(n-i) = \sum_{i=1}^{n} \frac{1}{n}(n-i) = \frac{1}{n}\sum_{i=1}^{n}(n-i) = \frac{1}{n}\cdot\frac{n(n-1)}{2} = \frac{n-1}{2}$$

因此，删除操作的平均时间复杂度为 $O(n)$。

按值查找（顺序查找）

在顺序表 L 中查找第一个值等于 e 的元素，返回其位序；若未找到，则返回 0。

Algorithm 5: LOCATEELEM(L, e)

Input: 顺序表 L，待查找元素 e
Output: 若找到返回元素位序（从 1 开始），否则返回 0

```
begin
    for i = 0; i < L.length; i++ do
        if L.data[i] == e then
            return i + 1
    return 0
```

最好情况： 目标元素在表头（$i = 1$），比较 1 次，时间复杂度为 $O(1)$。

最坏情况： 目标元素在表尾或不存在，需要比较 n 次，时间复杂度为 $O(n)$。

平均情况： 设目标元素在第 i 位的概率为 $p_i = 1/n$，则平均比较次数为

$$\sum_{i=1}^{n} p_i \cdot i = \sum_{i=1}^{n} \frac{1}{n}\cdot i = \frac{1}{n}\cdot\frac{n(n+1)}{2} = \frac{n+1}{2}$$

因此，按值查找的平均时间复杂度为 $O(n)$。

Question 2.1

有两个长度均为 n 的一维整型数组 A 和 res，对数组 A 中的每个元素 A[i]，计算 A[i] 与 A[j] $(0 \le i \le j \le n-1)$ 乘积的最大值，并将其保存到 res[i] 中。例如，当 A[] = {1, 4, -9, 6} 时，得到 res[] = {6, 24, 81, 36}。现给定数组 A，设计一个时间和空间上尽可能高效的算法 calMulMax，求 res 中各元素的值。函数原型为 `void calMulMax(int A[], int res[], int n)`。要求如下：

- 给出算法的基本设计思想。
- 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 说明你所设计算法的时间复杂度和空间复杂度。

1）算法的基本设计思想：

从后向前扫描一趟数组 A，对每个 $A[i]$ $(0 \le i \le n-1)$，分别找到从 $A[n-1]$ 到 $A[i]$ 中的最大值 Max 和最小值 Min，然后分以下情况进行处理：① 若 $A[i] \ge 0$，则 $A[i]$ 与 Max 相乘；② 若 $A[i] < 0$，则 $A[i]$ 与 Min 相乘。相乘的结果保存在 $res[i]$ 中。

2）算法实现：

Algorithm 6: CALMULMAX(A, res, n)

```
Input: 数组 A，长度 n
Output: 结果数组 res
begin
    Max, Min ⟵ A[n − 1]
    for i = n − 1; i ≥ 0; i − − do
        if A[i] > Max then
            Max ⟵ A[i]
        else if A[i] < Min then
            Min ⟵ A[i]
        if A[i] ≥ 0 then
            res[i] ⟵ A[i] × Max
        else
            res[i] ⟵ A[i] × Min
```

3）算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

2.3 线性表的链式表示

顺序表支持随机存取任意元素，但插入和删除操作需要移动大量元素，效率较低。相比之下，链式存储的线性表不要求地址连续的存储单元，即逻辑上相邻的元素在物理位置上是可以不相邻的；它通过指针建立元素之间的逻辑关系，因此插入或删除操作无须移动元素，而只需修改相关指针，效率较高。然而，这样做的代价是失去了顺序表的随机存取能力，只能从头开始顺序访问。

2.3.1 单链表的定义

Definition 2.3.1: 单链表

线性表的链式存储也称单链表。它通过一组任意的存储单元（不要求地址连续）来存储线性表中的数据元素。为了建立数据元素之间的线性关系，每个链表结点除了存放元素自身的信息外，还需额外设置一个指向其后继结点的指针。

单链表结点类型的定义如下：

Code 2.3.1 (单链表结点类型定义)

```
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
```

通常使用头指针 L（或 head 等）来标识一个单链表，该指针指向链表的起始位置。当头指针为 NULL 时，表示链表为空。此外，为了简化操作，在单链表的第一个数据结点之前常附加一个特殊的结点，称为头结点。头结点的数据域一般不存放有效数据（但可用于记录表长等信息）。在带头结点的单链表中，头指针 L 指向头结点，而头结点的 next 指向第一个数据结点，如图 2.1(a) 所示。在不带头结点的单链表中，头指针 L 直接指向第一个数据结点，如图 2.1(b) 所示。无论哪种形式，表尾结点的指针域为 NULL（图中用“∧”表示）。

(a) 带表头结点的单链表　　(b) 不带表头结点的单链表

图 2.1: 带头结点与不带头结点的单链表

头指针与头结点的关系：头指针始终指向链表的第一个结点（无论带不带头结点），而头结点仅存在于带头结点的链表中，它是链表的第一个结点，其数据域通常不存储实际数据。

引入头结点后，可带来以下两个主要优点：① **操作统一性**：第一个数据结点的位置存储在头结点的指针域中，因此在链表首部进行插入、删除等操作时，与其他位置的操作逻辑一致，无须特殊处理。② **空表与非空表处理统一**：无论链表是否为空，头指针始终是一个非空指针（指向头结点），而空表仅表现为头结点的 next 为 NULL，从而避免了对空表的单独判断。

2.3.2 单链表上基本操作的实现

单链表的初始化

带头结点与不带头结点的单链表在初始化时有所不同。带头结点的单链表初始化时，需创建一个头结点，并令头指针 L 指向该头结点，其 next 指针吃实话为 NULL。不带头结点的单链表初始化时，只需将头指针 L 初始化为 NULL。

Code 2.3.2 (带头结点的单链表的初始化)

```
bool InitList(LinkList &L) {
    L = (LNode*)malloc(sizeof(LNode));
    L->next = NULL;
    return true;
}
```

Code 2.3.3 (不带头结点的单链表的初始化)

```
bool InitList(LinkList &L) {
    L = NULL;
    return true;

}
```

Note:-

设 p 为指向链表结点的结构体指针，则 *p 表示该结点本身。可通过 p->data 或 (*p).data 访问其数据域，二者完全等价。成员运算符 (.) 左侧应为结构体变量，指向运算符 (->) 左侧应为结构体的指针。例如，p->next->data 等价于 (*(*p).next).data，表示当前访问结点后继结点的数据。

求表长操作

Algorithm 7: LENGTH(L)

```
Input: 单链表 L
Output: 链表长度 len
begin
    len ⟵ 0
    p ⟵ L
    while p->next ≠ NULL do
        p ⟵ p->next
        len ⟵ len + 1
    return len
```

求表长操作的时间复杂度为 $O(n)$。

按序号查找结点

Algorithm 8: GETELEM(L, i)

```
Input: 单链表 L，位序 i（从 1 开始）
Output: 指向第 i 个结点的指针，若不存在则返回 NULL
begin
    p ⟵ L
    j ⟵ 0
    while p ≠ NULL and j < i do
        p ⟵ p->next
        j ⟵ j + 1
    return p
```

按序号查找结点的时间复杂度为 $O(n)$。

按值查找表结点

Algorithm 9: LOCATEELEM(L, e)

```
Input: 单链表 L，待查找元素 e
Output: 指向第一个值为 e 的结点的指针，若不存在则返回 NULL
begin
    p ⟵ L->next
    while p ≠ NULL and p->data ≠ e do
        p ⟵ p->next
    return p
```

按值查找表结点的时间复杂度为 $O(n)$。

插入结点操作

先检查 i 的合法性，然后找到第 i-1 个结点（前驱），再在其后插入新结点。假设找到的第 i-1 个结点为 *p，然后令新结点 *s 的 next 指向 *p 的后继，再令 *p 的 next 指向 *s。

Algorithm 10: LISTINSERT(L, i, e)

Input: 单链表 L，插入位置 i（从 1 开始），待插入元素 e
Output: 插入成功返回 **true**，失败返回 **false**

```
begin
    p ⟵ L
    j ⟵ 0
    while p ≠ NULL and j < i − 1 do
        p ⟵ p−>next
        j ⟵ j + 1
    if p == NULL then
        return false
    s ⟵ (LNode *)malloc(sizeof(LNode))
    s−>data ⟵ e
    s−>next ⟵ p−>next
    p−>next ⟵ s
    return true
```

这里注意顺序不可颠倒。若先执行 $p->next = s$，则原后继地址丢失，导致 $s->next = p->next$ 实际变为 $s->next = s$，形成自环。该操作的时间复杂度为 $O(n)$，主要开销在于查找前驱结点。若已知某结点指针，则在其后插入新结点的时间复杂度为 $O(1)$。注意，当插入位置 $i = 1$ 时，不带头结点的单链表需要更新头指针 L 指向新首结点，而带头结点的则无须特殊处理.

扩展：对某个结点进行前插操作。

前插操作是指在某结点前插入一个新结点，与后插操作相反。任何前插操作均可转化为后插操作：只需从头结点开始顺序查找其前驱，时间复杂度为 $O(n)$。若对给定结点进行前插，则还可采用另一种技巧（适用于数据可复制的情况）：将新结点 s 插入目标结点 p 之后，再交换 p 与 s 的数据域。这种技巧逻辑上等效于前插，且时间复杂度为 $O(1)$。主要代码片段如下：

Code 2.3.4

```
s -> next = p -> next;
p -> next = s;
temp = p -> data;
p -> data = s -> data;
s -> data = temp;
```

删除结点操作

删除单链表的第 i 个数据结点。先检查 i 的合法性，然后找到第 i-1 个结点（前驱），再删除其后继结点，并释放内存。假设找到的第 i-1 个结点为 *p，其后继为被删结点 *q，先将 *p 的 next 指向 *q 的后继结点，然后释放 *q 的存储空间。

Algorithm 11: ListDelete(L, i, e)

Input: 单链表 L，删除位置 i（从 1 开始），变量 e 用于返回被删元素
Output: 删除成功返回 **true**，失败返回 **false**

```
begin
    p ⟵ L
    j ⟵ 0
    while p->next ≠ NULL and j < i − 1 do
        p ⟵ p->next
        j ⟵ j + 1
    if p->next == NULL or j > i − 1 then
        return false
    q ⟵ p->next
    e ⟵ q->data
    p->next ⟵ q->next
    free(q)
    return true
```

类似于插入操作，该操作的主要耗时也在查找操作上，时间复杂度为 $O(n)$。

当链表不带头结点时，需判断被删结点是否为首结点，若是，则要做特殊处理，将头指针 L 指向新的首结点。当链表带头结点时，删除首结点和删除其他结点的操作是相同的。

扩展：删除给定结点 *p。

常规方法需要从头遍历找到 *p 的前驱，时间复杂度为 $O(n)$。若允许修改结点内容，则可采用一种 $O(1)$ 的技巧：将 *p 的后继结点的数据复制到 *p 中，然后删除其后继结点。该方法的局限性在于不能用于删除尾结点（因其无后继）。主要代码片段如下：

Code 2.3.5

```
LNode *q = p -> next;
p -> data = p -> next -> data;
p -> next = q -> next;
free(q);
```

采用头插法建立单链表

Algorithm 12: List_HeadInsert(L)

Input: 输入整数序列，以 9999 结束
Output: 建立好的单链表 L

```
begin
    L ⟵ (LNode *)malloc(sizeof(LNode))
    L->next ⟵ NULL
    read(x)
    while x ≠ 9999 do
        s ⟵ (LNode *)malloc(sizeof(LNode))
        s->data ⟵ x
        s->next ⟵ L->next
        L->next ⟵ s
        read(x)
    return L
```

每插入一个结点的时间复杂度为 $O(1)$，总时间复杂度为 $O(n)$。

采用尾插法建立单链表

Algorithm 13: LIST_TAILINSERT(L)

Input: 输入整数序列，以 9999 结束
Output: 建立好的单链表 L

```
begin
    L ⟵ (LNode *)malloc(sizeof(LNode))
    s, r ⟵ L
    read(x)
    while x ≠ 9999 do
        s ⟵ (LNode *)malloc(sizeof(LNode))
        s->data ⟵ x
        r->next ⟵ s
        r ⟵ s
        read(x)
    r->next ⟵ NULL
    return L
```

因为附设了一个尾指针，故无需遍历查找尾部，总时间复杂度仍为 $O(n)$。

Note:-

在设计算法时，可先通过画图理清指针变化逻辑，再编写代码，有助于避免常见错误（如指针丢失、内存泄露等）。

2.3.3 双链表

Definition 2.3.2: 双链表

单链表的每个结点仅包含一个指向其后继的指针，故只能从前往后依次遍历。若需访问某结点的前驱（插入或删除操作时），则要从头开始遍历，时间复杂度为 $O(n)$。为了克服这一局限，引入了双链表，双链表中的每个结点包含两个指针 *prior* 和 *next*，分别指向直接前驱和直接后继，如图 2.2 所示。其中，头结点的 *prior* 为 NULL，尾结点的 *next* 也为 NULL。

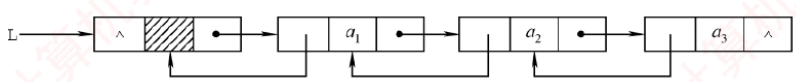

图 2.2: 双链表示意图

双链表结点类型定义如下：

Code 2.3.6 (双链表结点类型定义)

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinkList;
```

双链表的插入操作

在双链表的结点 *p 之后插入新结点 *s，其指针变化过程如图 2.3 所示。代码片段如下：

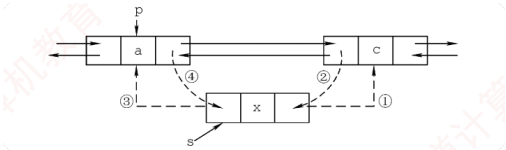

图 2.3: 双链表插入结点过程

Code 2.3.7 (双链表插入结点)

```
s -> next = p -> next;
p -> next -> prior = s;
s -> prior = p;
p -> next = s;
```

上述语句的执行顺序并不是任意的：步骤 ① 必须在步骤 ④ 之前，否则 $p->next$ 被覆盖后，无法访问原后继结点，导致指针丢失，插入失败。其余步骤可在保证逻辑正确的前提下适当调整（例如 ③ 可在 ① 前执行）。

双链表的删除操作

删除双链表中结点 *p 的后继结点 *q，其指针的变化过程如图 2.4 所示。代码片段如下：

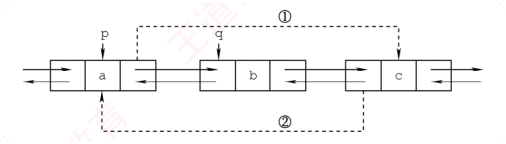

图 2.4: 双链表删除结点过程

Code 2.3.8 (双链表删除结点)

```
p -> next = q -> next;
q -> next -> prior = p;
free(q);
```

Note:-

建立双链表时，同样可以采用类似单链表的头插法或尾插法，但要注意：每次插入新节点时，要同时正确设置 next 和 prior 两个指针，以维持双链表的完整性。另外注意插入或删除时考虑边界情况，可以加入条件语句来处理特殊情况。

2.3.4 循环链表

循环单链表

Definition 2.3.3: 循环单链表

循环单链表与普通单链表的主要区别在于：表中最后一个结点的 next 域不再为 NULL，而是指向头结点，从而是整个链表形成一个环。这种结构是的从任意一个结点出发均可遍历整个循环单链表，而不仅限于从表头开始。

尾结点 $*r$ 的 next 指向头结点，因此链表中不存在 **next** 为 NULL 的结点。判空条件不再是检查头指针 L 是否为空，而是检查头结点的 next 是否指向自身（$L->next == L$）。

为了提高效率，循环单链表有时不设头指针，而仅设置尾指针 r。此时，在表头或表尾插入元素的时间复杂度均为 $O(1)$。若使用头指针，则表尾插入需要遍历整个链表，时间复杂度为 $O(n)$。例如，在表尾插入新结点 s 时，需要执行以下步骤：令 $s->next = r->next$（使新结点的后继指向头结点，以维持环状结构）；将 $r->next$ 指向新结点 s；更新 r 为新结点 s。

循环双链表

Definition 2.3.4: 循环双链表

循环双链表中的每个结点不仅包含指向下一个结点的 next 域，还包含指向前一个结点的 prior 域。特别地，头结点的 prior 需要指向表尾结点，从而形成一个完整的环形结构。

设尾结点为 *p，则 p->next 应指向头结点 L；同时，L->prior 应指向 *p。当循环双链表为空时，其头结点 L 的 prior 和 next 都指向自身（$L->prior = L$ 且 $L->next = L$）。

Question 2.2

设线性表 $L = (a_1, a_2, a_3, \cdots, a_{n-2}, a_{n-1}, a_n)$ 采用带头结点的单链表保存，链表中的结点定义如下：

```
typedef struct node {
    int data;
    struct node* next;
} NODE;
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法，重新排列 L 中的各结点，得到线性表 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \cdots)$。要求：

- 给出算法的基本设计思想。
- 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 说明你所设计的算法的时间复杂度。

1）算法的基本设计思想：

先观察 $L = (a_1, a_2, a_3, \cdots, a_{n-2}, a_{n-1}, a_n)$ 和 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \cdots)$，发现 L' 是由 L 摘取第一个元素，再摘取倒数第一个元素……依次合并而成的。为了方便链表后半段取元素，需要先将 L 的后半段原地逆置 [题目要求空间复杂度为 $O(1)$，不能借助栈]，否则每取最后一个结点都需要遍历一次链表。① 先找出链表 L 的中间结点，为此设置两个指针 p 和 q，指针 p 每次走一步，指针 q 每次走两步，当指针 q 到达链尾时，指针 p 正好在链表的中间结点；② 然后将 L 的后半段结点原地逆置；③ 从单链表前后两段中依次各取一个结点，按要求重排。

3）第一步找中间结点的时间复杂度为 $O(n)$，第二步逆置的时间复杂度为 $O(n)$，第三步合并链表的时间复杂度为 $O(n)$，所以该算法的时间复杂度为 $O(n)$。

2）算法实现：

Algorithm 14: CHANGE_LIST(h)

Input: 带头结点的单链表 h
Output: 重新排列后的单链表

```
begin
    p, q ⟵ h
    while q->next ≠ NULL do
        p ⟵ p->next
        q ⟵ q->next
        if q->next ≠ NULL then
            q ⟵ q->next
    q ⟵ p->next
    p->next ⟵ NULL
    while q ≠ NULL do
        r ⟵ q->next
        q->next ⟵ p->next
        p->next ⟵ q
        q ⟵ r
    s ⟵ h->next
    q ⟵ p->next
    p->next ⟵ NULL
    while q ≠ NULL do
        r ⟵ q->next
        q->next ⟵ s->next
        s->next ⟵ q
        s ⟵ q->next
        q ⟵ r
```

Chapter 3

栈与队列

3.1 栈

3.1.1 栈的基本概念

Definition 3.1.1: 栈

栈（Stack）是仅允许在一端进行插入和删除操作的线性表。作为一种特殊的线性表，栈的插入（入栈）和删除（出栈）操作被限制在表的一端进行。栈的操作特性可概括为后进先出（Last In First Out, LIFO）。

Definition 3.1.2: 栈顶，栈底，空栈

- 栈顶（Top）：允许进行插入和删除操作的一端。
- 栈底（Bottom）：固定不变、不允许进行插入和删除操作的另一端。
- 空栈（Empty Stack）：不含任何元素的栈。

栈的基本操作

- **InitStack (&S)**：初始化一个空栈 S。
- **StackEmpty (S)**：判断栈是否为空。若栈 S 为空，返回 true；否则返回 false。
- **Push (&S, x)**：入栈操作；若栈未满，则 x 成为新的栈顶元素。
- **Pop (&S, &x)**：出栈操作；若栈非空，则弹出栈顶元素，并通过 x 返回该值。
- **GetTop (S, &x)**：读取栈顶元素但不出栈；若栈非空，则通过 x 返回栈顶元素。
- **DestroyStack (&S)**：销毁栈 S，并释放其所占用的存储空间（& 表示引用调用）。

3.1.2 栈的顺序存储结构

与线性表类似，栈也有两种基本的存储方式：顺序存储和链式存储。

顺序栈的实现

Definition 3.1.3: 顺序栈

采用顺序存储的栈称为顺序栈，它利用一组地址连续的存储单元存放从栈底到栈顶的数据元素，并附设一个整型指针（top）指示当前栈顶元素的位置。

顺序栈的类型可定义如下：

Code 3.1.1

```
#define MaxSize 50
typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
```

栈顶指针：S.top，初始时设为 S.top=-1；栈顶元素：S.data[S.top]。
入栈操作：当栈未满时，先将栈顶指针加 1，再将元素存入栈顶。
出栈操作：当栈非空时，先取出栈顶元素，再将栈顶指针减 1。
栈空条件：S.top==-1；栈满条件：S.top==MaxSize-l；栈长：S.top+l。

顺序栈的基本操作

以下是顺序栈常用基本操作的实现（基于 top = -1 初始化）。
（1）初始化

Code 3.1.2

```
void InitStack(SqStack &S) {
    S.top = -1;
}
```

（2）判栈空

Algorithm 15: STACKEMPTY(S)

Input: 顺序栈 S
Output: 栈空返回 **true**，否则返回 **false**

```
begin
    if S.top == −1 then
        return true
    else
        return false
```

（3）入栈

Algorithm 16: PUSH(S, x)

Input: 顺序栈 S，待入栈元素 x
Output: 入栈成功返回 **true**，失败返回 **false**

```
begin
    if S.top == MaxSize − 1 then
        return false
    S.data[++S.top] ⟵ x
    return true
```

（4）出栈

Algorithm 17: POP(S, x)

Input: 顺序栈 S，变量 x 用于返回栈顶元素
Output: 出栈成功返回 **true**，失败返回 **false**

```
begin
    if S.top == −1 then
        return false
    x ⟵ S.data[S.top−−]
    return true
```

（5）读栈顶元素

Algorithm 18: GETTOP(S, x)

Input: 顺序栈 S，变量 x 用于返回栈顶元素
Output: 获取成功返回 **true**，失败返回 **false**

```
begin
    if S.top == −1 then
        return false
    x ⟵ S.data[S.top]
    return true
```

Note:-

此处，top 指向栈顶元素本身。因此入栈为 S.data[++S.top] = x，出栈为 x = S.data[S.top−]。若 top 初始化为 0（指向栈顶元素的下一个位置），则入栈变为 S.data[S.top++] = x，出栈变为 x = S.data[−S.top]，相应的判空、判满条件也随之改变。

subsection 栈的链式存储结构

Definition 3.1.4: 链栈

采用链式存储的栈称为链栈。其优点是便于动态分配存储空间，不存在栈满溢出的问题，且在多栈共存的场景下能更灵活地利用内存。通常使用单链表实现链栈，并且规定所有操作均在链表的表头进行。此处约定链栈不带头结点，栈顶指针 Lhead 直接指向栈顶元素。

链栈的类型定义如下：

Code 3.1.3

```c
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LiStack;
```

Question 3.1

设单链表的表头指针为 L，结点结构由 data 和 next 两个域构成，其中 data 域为字符型。设计算法判断该链表的全部 n 个字符是否中心对称。例如，xyx、xyyx 都是中心对称的。

算法思想：使用栈来判断链表中的数据是否中心对称。让链表的前一半元素依次入栈。在处理链表的后一半元素时，当访问到链表的一个元素后，就从栈中弹出一个元素，对两个元素进行比较，若相等，则将链表中的下一个元素与栈中再弹出的元素进行比较，直至链表到尾。这时若栈是空栈，则得出链表中心对称的结论；否则，当链表中的一个元素与栈中弹出元素不等时，结论为链表非中心对称，结束算法的执行.

算法如下：

Algorithm 19: DC(L, n)

```
Input: 带头结点的单链表 L，链表长度 n
Output: 若链表中心对称返回 1，否则返回 0
begin
    char s[n/2]
    p ⟵ L->next
    for i = 0; i < n/2; i++ do
        s[i] ⟵ p->data
        p ⟵ p->next
    i--
    if n%2 == 1 then
        p ⟵ p->next
    while p ≠ NULL and s[i] == p->data do
        i--
        p ⟵ p->next
    if i == -1 then
        return 1
    else
        return 0
```

算法先将“链表的前一半”元素（字符）入栈。当 n 为偶数时，前一半和后一半的个数相同；当 n 为奇数时，链表中心结点字符不必比较，移动链表指针到下一字符开始比较。比较过程中遇到不相等时，立即退出 while 循环，不再进行比较。

3.2 队列

3.2.1 队列的基本概念

Definition 3.2.1: 队列

队列（Queue）简称队，也是一种操作受限的线性表，仅允许在表的一端进行插入，而在另一端进行删除。向队列中插入元素称为入队（或进队），删除元素称为出队（或离队）。队列的操作特性为先进先出（First In First Out, FIFO）。

Definition 3.2.2: 队头，队尾，空队列

- 队头（Front）：允许进行删除操作的一端，也称队首。
- 队尾（Rear）：允许进行插入操作的一端。
- 空队列（Empty Queue）：不含任何元素的队列。

队列的基本操作

- **InitQueue (&Q)**：初始化队列，构造一个空队列 Q。
- **QueueEmpty (Q)**：判队列空；若队列 Q 为空，返回 true，否则返回 false。
- **EnQueue (&Q, x)**：入队操作；若队列 Q 未满，将 x 加入队尾。
- **DeQueue (&Q, &x)**：出队操作；若队列 Q 非空，删除队首元素，并通过 x 返回其值。
- **GetHead (Q, &x)**：读队首元素但不出队；若队列 Q 非空，将队首元素赋值给 x。

Note:-

栈和队列均为操作受限的线性表，因此并非所有线性表的操作都适用于它们。例如，不允许直接访问或修改栈或队列中间的元素，这是由其逻辑特性所决定的。

3.2.2 队列的顺序存储结构

队列的顺序存储

队列的顺序实现是指分配一块连续的存储单元存放队列元素，并附设两个指针：队首指针 front 指向队首元素，队尾指针 rear 指向队尾元素的下一个位置。队列的顺序存储可定义为：

Code 3.2.1

```c
#define MaxSize 50
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```

初始状态：Q.front = Q.rear = 0。

入队操作：当队列未满时，先将元素存入队尾，再将队尾指针加 1。

出队操作：当队列非空时，先取出队首元素，再将队首指针加 1。

然而，能否以 Q.rear==MaxSize 作为队满条件呢？显然不能。当队列中仅剩一个元素，但 rear 已达 MaxSize，若此时继续入队，则会出现“上溢”。但这种溢出并非真正溢出，实际上数组中仍有空闲单元，因此称此现象为假溢出。

循环队列

为了解决顺序队列的假溢出问题，引入循环队列：将顺序存储空间在逻辑上组织为环状结构。当指针到达数组末尾时，自动回到起始位置，这可通过取模运算（%）实现。

初始状态：Q.front = Q.rear = 0。

队首指针进 1：Q.front=(Q.front+l)%MaxSize。

队尾指针进 1：Q.rear=(Q.rear+l)%MaxSizeo

队列长度：(Q.rear+MaxSize-Q.front)%MaxSize。

出入队操作：指针均按顺时针方向移动。

那么，循环队列判空和判满的条件是什么呢？队空条件显然为 Q.front==Q.rear。但当入队速度远快于出队时，rear 会追上 front，此时同样满足 Q.front==Q.rear，却表示队满。因此，仅凭 front==rear 无法区分队空与队满。循环队列出入队的示意图如图 3.1 所示。

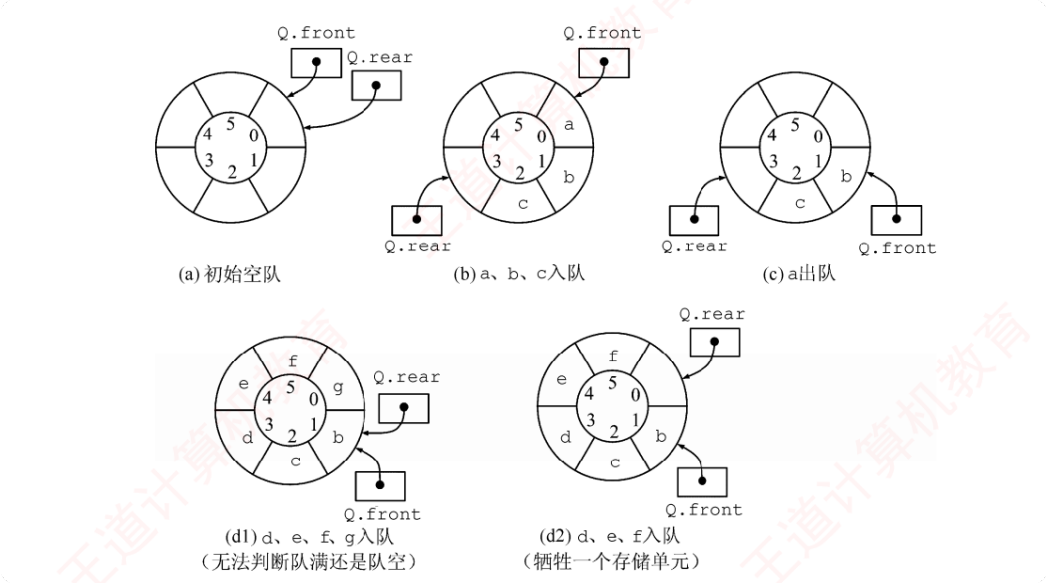

图 3.1: 循环队列出入队示意图

为解决这一问题，常用以下三种方法：

1）牺牲一个存储单元：入队时少用一个存储单元，这是一种较为普遍的做法，约定以“队尾指针的下一位置为队首”作为队满标志，如图 3.1(d2) 所示。

队满条件：(Q.rear+l)%MaxSize==Q.front。

队空条件：Q.front==Q.rear。

队列中元素个数：(Q.rear-Q.front+MaxSize)%MaxSize。

2）增设 size 成员：记录当前元素个数。若入队成功，则 size++，若出队成功，则 size−。

队空条件：Q.size==0。

队满条件：Q.size==MaxSize。

两种情形均有 Q.front==Q.rear，由 size 区分队空与队满。

3）增设 tag 标志位：

出队后置 tag=0，若此时 Q.front==Q.rear，**则为队空**。

入队后置 tag=l，若此时 Q.front==Q.rear，**则为队满**。

循环队列的操作（基于牺牲一个单元法）

（1）初始化

Code 3.2.2

```
void InitQueue(SqQueue &Q) {
    Q.rear = Q.front = 0;
}
```

（2）判队空

Algorithm 20: QueueEmpty(Q)

Input: 循环队列 Q
Output: 队列空返回 **true**，否则返回 **false**

```
begin
    if Q.rear == Q.front then
        return true
    else
        return false
```

（3）入队

Algorithm 21: EnQueue(Q, x)

Input: 循环队列 Q，待入队元素 x
Output: 入队成功返回 **true**，失败返回 **false**

```
begin
    if (Q.rear + 1)%MaxSize == Q.front then
        return false
    Q.data[Q.rear] ⟵ x
    Q.rear ⟵ (Q.rear + 1)%MaxSize
    return true
```

（4）出队

Algorithm 22: DeQueue(Q, x)

Input: 循环队列 Q，变量 x 用于返回队头元素
Output: 出队成功返回 **true**，失败返回 **false**

```
begin
    if Q.rear == Q.front then
        return false
    x ⟵ Q.data[Q.front]
    Q.front ⟵ (Q.front + 1)%MaxSize
    return true
```

3.2.3 队列的链式存储结构

队列的链式存储

Definition 3.2.3: 链式队列

队列的链式表示称为链式队列，本质上是一个同时带有队首指针和队尾指针的单链表。队首指针指向队头结点，队尾指针指向队尾结点，即单链表的最后一个结点。

链式队列的类型可定义为：

Code 3.2.3

```
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode;
typedef struct {
    LinkNode *front, *rear;
} LinkQueue;
```

当不带头结点时，若 Q.front==NULL 且 Q.rear==NULL，则链式队列为空队列。

链式队列的基本操作

（1）初始化

Code 3.2.4

```
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (LinkNode *)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
```

（2）判队空

Algorithm 23: QueueEmpty(Q)

Input: 链式队列 Q
Output: 队列空返回 **true**，否则返回 **false**

```
begin
    if Q.front == Q.rear then
        return true
    else
        return false
```

（3）入队

Algorithm 24: EnQueue(Q, x)

Input: 链式队列 Q，待入队元素 x
Output: 无返回值

```
begin
    s ⟵ (LinkNode *)malloc(sizeof(LinkNode))
    s->data ⟵ x
    s->next ⟵ NULL
    Q.rear->next ⟵ s
    Q.rear ⟵ s
```

（4）出队

Algorithm 25: DeQueue(Q, x)

Input: 链式队列 Q，变量 x 用于返回队头元素
Output: 出队成功返回 **true**，失败返回 **false**

```
begin
    if Q.front == Q.rear then
        return false
    p ⟵ Q.front->next
    x ⟵ p->data
    Q.front->next ⟵ p->next
    if Q.rear == p then
        Q.rear ⟵ Q.front
    free(p)
    return true
```

Question 3.2

Q 是一个队列，S 是一个空栈，实现将队列中的元素逆置的算法。

Algorithm 26: Inverser(S, Q)

Input: 空栈 S，队列 Q
Output: 队列 Q 中的元素被逆置

```
begin
    while not QueueEmpty(Q) do
        x ⟵ DeQueue(Q)
        Push(S, x)
    while not StackEmpty(S) do
        Pop(S, x)
        EnQueue(Q, x)
```

3.3 栈和队列的应用

3.3.1 栈在括号匹配中的应用

Algorithm 27: bracketCheck(str, len)

Input: 字符数组 $str[]$，数组长度 len
Output: 括号匹配返回 **true**，否则返回 **false**

```
begin
    SqStack S
    InitStack(S)
    for i = 0; i < len; i++ do
        if str[i] =='(' or str[i] =='[' or str[i] =='{' then
            Push(S, str[i])
        else if str[i] ==')' or str[i] ==']' or str[i] =='}' then
            if StackEmpty(S) then
                return false
            top ⟵ Pop(S)
            if (str[i] ==')' and top! ='(') or (str[i] ==']' and top! ='[') or (str[i] ==='}' and top! ='{')
             then
                return false
    return StackEmpty(S)
```

3.3.2 栈在表达式求值中的应用

表 3.1: 中缀表达式 $A+B*(C-D)-E/F$ 转后缀表达式的过程

步	待处理序列	栈内	后缀表达式	扫描项	说明
1	$A+B*(C-D)-E/F$			A	A 加入后缀表达式
2	$+B*(C-D)-E/F$		A	$+$	+ 入栈
3	$B*(C-D)-E/F$	$+$	A	B	B 加入后缀表达式
4	$*(C-D)-E/F$	$+$	AB	$*$	* 优先级高于栈顶，* 入栈
5	$(C-D)-E/F$	$+*$	AB	(	(直接入栈)
6	$C-D)-E/F$	$+*($	AB	C	C 加入后缀表达式
7	$-D)-E/F$	$+*($	ABC	$-$	栈顶为 (，-直接入栈
8	$D)-E/F$	$+*(-$	ABC	D	D 加入后缀表达式
9	$)-E/F$	$+*(-$	$ABCD$	)	遇到)，弹出-，删除 (
10	$-E/F$	$+*$	$ABCD-$	$-$	-优先级低于栈顶，依次弹出 *、+、-入栈
11	E/F	$-$	$ABCD-*+$	E	E 加入后缀表达式
12	$/F$	$-$	$ABCD-*+E$	$/$	/优先级高于栈顶，/入栈
13	F	$-/$	$ABCD-*+E$	F	F 加入后缀表达式
14		$-/$	$ABCD-*+EF$		扫描结束，弹出剩余运算符
15			$ABCD-*+EF/-$		结束

表 3.2: 后缀表达式 $ABCD-*+EF/-$ 的求值过程

步	扫描项	项类型	动作	栈中内容
1			置空栈	空
2	A	操作数	入栈	A
3	B	操作数	入栈	A B
4	C	操作数	入栈	A B C
5	D	操作数	入栈	A B C D
6	-	操作符	D、C 出栈，计算 $C-D$，结果 R_1 入栈	A B R_1
7	*	操作符	R_1、B 出栈，计算 $B\times R_1$，结果 R_2 入栈	A R_2
8	+	操作符	R_2、A 出栈，计算 $A+R_2$，结果 R_3 入栈	R_3
9	E	操作数	入栈	R_3 E
10	F	操作数	入栈	R_3 E F
11	/	操作符	F、E 出栈，计算 E/F，结果 R_4 入栈	R_3 R_4
12	-	操作符	R_4、R_3 出栈，计算 R_3-R_4，结果 R_5 入栈	R_5

Question 3.3

设计一个栈，使它可以在 $O(1)$ 的时间复杂度内实现 Push、Pop 和 min 操作。所谓 min 操作, 是指得到栈中最小的元素。

Note:-

使用双栈，两个栈是同步关系。主栈是普通栈，用来实现栈的基本操作 Push 和 Pop；辅助栈用来记录同步的最小值 min，例如元素 x 入栈，则辅助栈 stack_min[top++] = $(x<\text{min})?x:\text{min}$；即在每次 Push 中，都将当前最小元素放到 stack_min 的栈顶。在主栈中 Pop 最小元素 y 时，stack_min 栈中相同位置的最小元素 y 也会随着 top−− 而出栈。因此 stack_min 的栈顶元素必然是之前入栈的最小元素。这是典型的以空间换时间的算法。

Chapter 4

串

4.1 串的定义和实现

4.1.1 串的定义

Definition 4.1.1: 串

串（string）是由零个或多个字符组成的有限序列，通常记为

$$S =' a_1 a_2 \cdots a_n' \quad (n \geq 0)$$

其中，S 是串名，单引号括起的字符序列是串的值；每个 a_i 可以是字母、数字或其他字符；串中字符的个数 n 称为串的长度。当 $n = 0$ 时，该串称为空串（用 $\varnothing$ 表示）。

4.1.2 串的基本操作

- **StrAssign(&T, chars)**：赋值操作。将串 T 赋值为 *chars*。
- **StrCopy(&T, S)**：复制操作。将串 S 复制给串 T。
- **StrEmpty(S)**：判空操作。若 S 为空串，则返回 TRUE，否则返回 FALSE。
- **StrCompare(S, T)**：比较操作。若 $S > T$，返回值 > 0；若 $S = T$，返回值 $= 0$；若 $S < T$，返回值 < 0。
- **StrLength(S)**：求串长操作。返回串 S 中字符的个数。
- **SubString(&Sub, S, pos, len)**：求子串操作。用 *Sub* 返回串 S 中从第 *pos* 个字符起、长度为 *len* 的子串。
- **Concat(&T, S1, S2)**：串联操作。用 T 返回由 $S1$ 和 $S2$ 联接而成的新串。
- **Index(S, T)**：定位操作。若主串 S 中存在与串 T 值相同的子串，则返回其在 S 中首次出现的位置；否则返回 0。
- **ClearString(&S)**：清空操作。将串 S 置为空串。
- **DestroyString(&S)**：销毁操作。将串 S 销毁（& 表示引用调用）。

Algorithm 28: SUBSTRING(Sub, S, pos, len)

Input: 主串 S，起始位置 pos（从 1 开始），子串长度 len
Output: 子串 Sub，操作成功返回 **true**，失败返回 **false**

```
begin
    if pos + len − 1 ≥ S.length then
        return false
    for i = pos; i < pos + len; i++ do
        Sub.ch[i − pos + 1] ⟵ S.ch[i]
    Sub.length ⟵ len
    return true
```

Algorithm 29: STRCOMPARE(S, T)

Input: 串 S，串 T
Output: 比较结果：$S > T$ 返回正数，$S = T$ 返回 0，$S < T$ 返回负数

```
begin
    for i = 1; i ≤ S.length and i ≤ T.length; i++ do
        if S.ch[i] ≠ T.ch[i] then
            return S.ch[i] − T.ch[i]
    return S.length − T.length
```

Algorithm 30: INDEX(S, T)

Input: 主串 S，模式串 T
Output: T 在 S 中首次出现的位置，不存在则返回 0

```
begin
    i ⟵ 1, n ⟵ S.length, m ⟵ T.length
    while i ≤ n − m + 1 do
        SubString(sub, S, i, m)
        if StrCompare(sub, T) == 0 then
            return i
        i ⟵ i + 1
    return 0
```

4.1.3 串的存储结构

定长顺序存储表示

Code 4.1.1

```c
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN];
    int length;
} SString;
```

串的实际长度不得超过 MAXLEN，若超出，则多余部分被舍弃，称为截断。串长有两种表示方式：一种是如上述结构所示，用一个独立的整型变量 len 显式记录串的长度；另一种是在串值末尾添加一个不计入串长的结束标记符 `'\0'`，此时串长为隐含值，需要通过遍历计算得出。

堆分配存储表示

堆分配存储仍以地址连续的存储单元存放串值字符序列，但其存储空间在程序运行过程中动态申请获得。

Code 4.1.2

```
typedef struct {
    char *ch;
    int length;
} HString;
```

在 C 语言中，存在一个称为堆的自由存储区。可通过调用 `malloc()` 为新生成的串动态分配一块大小等于串长的连续存储空间：若分配成功，返回指向该空间起始地址的指针，作为串的基地址（由指针 ch 指示）；若分配失败，则返回 NULL。已分配的空间可通过 `free()` 释放。

4.2 串的模式匹配

4.2.1 简单的模式匹配算法

Algorithm 31: IndexString(S, T)

Input: 主串 S，模式串 T
Output: T 在 S 中首次出现的位置，不存在则返回 0

```
begin
    i ⟵ 1, j ⟵ 1
    while i ≤ S.length and j ≤ T.length do
        if S.ch[i] == T.ch[j] then
            i ⟵ i + 1
            j ⟵ j + 1
        else
            i ⟵ i − j + 2
            j ⟵ 1
    if j > T.length then
        return i − T.length
    return 0
```

4.2.2 串的模式匹配算法——KMP 算法

在简单模式匹配中，每次匹配失败后都会将模式串向右滑动一位并从头开始比较。但是，某次已匹配成功的序列实际上是模式串的某个前缀，通过分析模式串本身的结构，**若已匹配成功的序列中有后缀正好是模式串的前缀，则可直接将模式串右滑到与这些字符对齐的位置，而无须回溯主串指针 i**，从而提高效率。这种优化策略正是 KMP 算法的核心。

KMP 算法的原理和 next 数组的手动求解参考 B 站视频：KMP 算法。

KMP 算法的进一步优化参考 B 站视频：KMP 算法的进一步优化。

KMP 算法的实现

求解 next 数组的算法如下：

Algorithm 32: GET_NEXT($T, next$)

```
Input: 模式串 T
Output: next 数组
begin
    i ⟵ 1, j ⟵ 0
    next[1] ⟵ 0
    while i < T.length do
        if j == 0 or T.ch[i] == T.ch[j] then
            i ⟵ i + 1
            j ⟵ j + 1
            next[i] ⟵ j
        else
            j ⟵ next[j]
```

KMP 算法实现如下：

Algorithm 33: INDEX_KMP($S, T, next$)

```
Input: 主串 S，模式串 T，next 数组
Output: T 在 S 中首次出现的位置，不存在则返回 0
begin
    i ⟵ 1, j ⟵ 1
    while i ≤ S.length and j ≤ T.length do
        if j == 0 or S.ch[i] == T.ch[j] then
            i ⟵ i + 1
            j ⟵ j + 1
        else
            j ⟵ next[j]
    if j > T.length then
        return i − T.length
    return 0
```

Chapter 5

树与二叉树

5.1 树的基本概念

5.1.1 树的定义

Definition 5.1.1: 树

树是 n $(n \geq 0)$ 个结点的有限集。当 $n = 0$ 时，称为空树。在任意一棵非空树中必须满足：

1）有且仅有一个特定的结点称为根结点。

2）当 $n > 1$ 时，其余结点可分为 m $(m > 0)$ 个互不相交的有限集 $T_1, T_2, \cdots, T_m$，其中每个集合本身又是一棵树，称为根的子树。

显然，树的定义是递归的——在定义中引用了自身，因此树是一种典型的递归数据结构。作为一种逻辑结构，树同时也是一种分层结构，具有以下两个特征：

1）根结点没有前驱，除根结点外的每个结点有且仅有一个前驱。

2）所有结点均可拥有零个或多个后继。

5.1.2 树的性质

树具有如下基本性质：

- 树的结点数 n 等于所有结点的度数之和加 1。
- 度为 m 的树中，第 i 层上至多有 m^{i-1} 个结点 $(i \geq 1)$。
- 高度为 h 的 m 叉树至多有 $(m^h - 1)/(m - 1)$ 个结点。
- 度为 m，具有 n 个结点的树的最小高度为 $\lceil \log_m(n(m-1)+1) \rceil$。
- 度为 m，具有 n 个结点的树的最大高度为 $n - m + 1$。

Note:-

常用于求解树结点与度之间关系的有：

- 总结点数 $= n_0 + n_1 + n_2 + \cdots + n_m$，其中 n_i 表示度为 i 的结点数。
- 总分支数 $= 1 \cdot n_1 + 2 \cdot n_2 + \cdots + m \cdot n_m$（度为 m 的结点引出 m 条分支）。
- 总结点数 = 总分支数 +1。

5.2 二叉树的概念

5.2.1 二叉树的定义及其主要特性

二叉树的定义

Definition 5.2.1: 二叉树

二叉树是一种特殊的树形结构，其核心特征在于：每个结点至多拥有两棵子树（不存在度大于 2 的结点），且这两棵子树具有明确的左右之分，其次序是固定的，不可交换。

与树类似，二叉树也以递归的方式定义。二叉树是包含 n ($n \geq 0$) 个结点的有限集合：

1. 或者为空二叉树，即 $n = 0$。

2. 或者由一个根结点以及两棵互不相交的被称为根的左子树和右子树构成，且左右子树本身均为二叉树。

二叉树属于有序树。即使某个结点仅有一棵子树，也要区分它是左子树还是右子树；若交换某结点的左右子树，则会得到另一棵不同的二叉树。

Note:-

注意，二叉树与度为 2 的有序树有本质区别：

- 度为 2 的树至少包含 3 个结点，而二叉树可以为空。
- 在度为 2 的有序树中，若某结点只有一个孩子，则无须区分左右次序；而在二叉树中，无论结点是否拥有两个孩子，其子树的左右位置都是确定且不可省略的，这是结构本身的固有属性。

几种特殊的二叉树

1）满二叉树。一颗高度为 h 且包含 $2^h - 1$ 个结点的二叉树称为满二叉树，即每一层都含有该层所能容纳的最大结点数。满二叉树的所有叶结点均位于最下一层，且除叶结点外一，其余每个结点的度均为 2。

2）完全二叉树。高度为 h 且包含 n 个结点的二叉树，当且仅当其每个结点与高度为 h 的满二叉树中编号为 1 n 的结点一一对应时，称为完全二叉树。

3）二叉排序树。其左子树上所有结点的关键字均小于根节点的关键字；右子树所有结点的关键字均大于根节点的关键字，且左子树和右子树本身也各是一个二叉排序树。

4）平衡二叉树。树中任意一个结点的左子树和右子树的高度之差的绝对值不超过 1。

5）正则二叉树。树中每个分支结点均有 2 个孩子，即树中仅有度为 0 或 2 的结点。

二叉树的性质

1）非空二叉树的叶结点数等于度为 2 的结点数加 1，即 $n_0 = n_2 + 1$。

2）非空二叉树的第 k 层最多有 2^{k-1} 个结点 ($k \geq 1$)。

3）高度为 h 的二叉树至多有 $2^h - 1$ 个结点 ($h \geq 1$)。

4）对完全二叉树按从上到下、从左到右的顺序依次编号 $1, 2, \ldots, n$，则有以下关系：

- 最后一个分支结点的编号为 $\lfloor n/2 \rfloor$，若 $i \leq \lfloor n/2 \rfloor$，则结点 i 为分支结点，否则为叶结点。
- 叶结点只可能出现在最后两层上（相当于在相同高度的满二叉树的最底层、最右侧减少一些连续叶结点；当减少两个及以上叶结点时，次底层将出现叶结点）。

- 若存在度为 1 的结点，则最多只可能有一个，且该结点只有左孩子而无右孩子（度为 1 的分支结点只可能是最后一个分支结点，其结点编号为 $\lfloor n/2 \rfloor$）。
- 按层序编号后，一旦某个结点 i 为叶结点或仅有左孩子，则所有编号大于 i 的结点均为叶结点（与上述结论 ① 和结论 ③ 一致）。
- 若 n 为奇数，则每个分支结点都有左右孩子；若 n 为偶数，则编号最大的分支结点（编号为 $n/2$）只有左孩子，没有右孩子，其余分支结点都有左右孩子。
- 当 $i > 1$ 时，结点 i 的双亲结点的编号为 $\lfloor i/2 \rfloor$。
- 若结点 i 有左右孩子，则左孩子编号为 $2i$，右孩子编号为 $2i+1$。
- 结点 i 所在层次（深度）为 $\lfloor \log_2 i \rfloor + 1$。

5）具有 n 个（$n > 0$）结点的完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$。

设高度为 h，根据性质 3 和完全二叉树的定义有

$$2^{h-1} - 1 < n \le 2^h - 1 \quad \text{或者} \quad 2^{h-1} \le n < 2^h$$

得出 $2^{h-1} < n+1 \le 2^h$，即 $h - 1 < \log_2(n+1) \le h$，因为 h 为正整数，所以 $h = \lceil \log_2(n+1) \rceil$，或者得出 $h - 1 \le \log_2 n < h$，所以 $h = \lfloor \log_2 n \rfloor + 1$。

5.2.2 二叉树的存储结构

链式存储结构

由于顺序存储的空间利用率较低，二叉树通常采用链式存储结构。在二叉树的链式存储中，每个结点通常包括若干数据域和指针域。二叉树结点至少包含 3 个域：左指针域 `lchild`、数据域 `data` 和右指针域 `rchild`。还可根据需要扩展结点结构，例如增加指向父结点的指针域，从而形成三叉链表的存储结构。

二叉树的链式存储结构可定义如下：

Code 5.2.1

```
typedef struct BitNode {
    ElemType data;
    struct BitNode *lchild, *rchild;
} BitNode, *BiTree;
```

Question 5.1

1. 若三叉树 T 中有 244 个结点（叶结点的高度为 1），则 T 的高度至少是 ()。

A. 8　　B. 7　　C. 6　　D. 5

2. 若二叉树的结点值均为正整数，采用顺序存储方式保存在数组 R 中，用 -1 表示结点不存在，则下列数组中，不能表示一棵二叉树的是 ()。

A. $R[] = \{20, 15, 40, -1, -1, 35\}$

B. $R[] = \{15, 40, 10, 18, 35, -1, -1\}$

C. $R[] = \{15, 40, 10, -1, -1, -1, 12\}$

D. $R[] = \{17, 20, 35, -1, 18, 45, -1, -1, 19, 27\}$

1. C

高度一定的三叉树中结点数最多的情况是满三叉树。高度为 5 的满三叉树的结点数 $= 3^0 + 3^1 + 3^2 + 3^3 + 3^4 = 121$，高度为 6 的满三叉树的结点数 $= 3^0 + 3^1 + 3^2 + 3^3 + 3^4 + 3^5 = 364$。三叉树 T 的结点数为 244，$121 < 244 < 364$，因此 T 的高度至少为 6。

2. D

在二叉树的顺序存储结构中，结点按完全二叉树的层次顺序存放：根在下标 0，对于任意非根结点 $R[i]$，其父结点为 $R[(i-1)/2]$。若某结点为空，则其所有后代位置必须也为空，否则将出现“空结点拥有非空子结点”的情况，违反二叉树的定义。在选项 A、B、C 中，所有非 -1 的元素，其从根到该结点的祖先路径上均不含 -1，符合顺序存储规则。在选项 D 中，$R[8] = 19$ 是一个有效结点，其父结点应为 $R[(8-1)/2] = R[3]$，但 $R[3] = -1$，表示该结点不存在，因此无法构成合法的二叉树。

5.3 二叉树的遍历和线索二叉树

5.3.1 二叉树的遍历

先序遍历（PreOrder）

若二叉树为空，则不执行任何操作；否则，1）访问根节点；2）先序遍历左子树；3）先序遍历右子树。对应的递归算法如下：

Algorithm 34: PreOrder(T)

```
Input: 二叉树 T
Output: 先序遍历序列
begin
    if T ≠ NULL then
        visit(T)
        PreOrder(T->lchild)
        PreOrder(T->rchild)
```

中序遍历（InOrder）

若二叉树为空，则不执行任何操作；否则，1）中序遍历左子树；2）访问根节点；3）中序遍历右子树。对应的递归算法如下：

Algorithm 35: InOrder(T)

```
Input: 二叉树 T
Output: 中序遍历序列
begin
    if T ≠ NULL then
        InOrder(T->lchild)
        visit(T)
        InOrder(T->rchild)
```

后序遍历（PostOrder）

若二叉树为空，则不执行任何操作；否则，1）后序遍历左子树；2）后序遍历右子树；3）访问根节点。对应的递归算法如下：

Algorithm 36: PostOrder(T)

```
Input: 二叉树 T
Output: 后序遍历序列
begin
    if T ≠ NULL then
        PostOrder(T->lchild)
        PostOrder(T->rchild)
        visit(T)
```

不论采用哪种遍历方法，每个结点都被访问一次且仅一次，时间复杂度均为 $O(n)$。在递归实现中，系统工作栈的深度等于树的高度，在最坏情况下（如单支树），空间复杂度为 $O(n)$。

层次遍历（LevelOrder）

层次遍历是按照树的层次从上到下、从左到右依次访问每个结点的遍历方法。层次遍历需要借助一个队列，其基本思想如下：首先将根节点入队，随后不断出队并访问对头结点，同时将其存在的左右孩子依次加入队尾，直至队列为空。

二叉树的层次遍历算法如下：

Algorithm 37: LevelOrder(T)

```
Input: 二叉树 T
Output: 层次遍历序列
begin
    InitQueue(Q)
    EnQueue(Q, T)
    while not IsEmpty(Q) do
        DeQueue(Q, p)
        visit(p)
        if p->lchild ≠ NULL then
            EnQueue(Q, p->lchild)
        if p->rchild ≠ NULL then
            EnQueue(Q, p->rchild)
```

5.3.2 线索二叉树

线索二叉树的基本概念

Definition 5.3.1: 线索二叉树

传统的二叉链表结构仅能反映父子关系，无法直接获取结点在某种遍历序列中的前驱或后继。前面提到，在含有 n 个结点的二叉树中，共有 $n+1$ 个空指针域：每个叶结点贡献 2 个空指针，每个度为 1 的结点贡献 1 个空指针，因此空指针总数为 $2n_0+n_1$，又 $n_0=n_2+1$，可得空指针总数为 $n_0+n_1+n_2+1=n+1$。可否利用这些空指针，直接指向结点在遍历序列中的前驱或后继？若能实现，便可使遍历链表一样高效地遍历二叉树，线索二叉树正是基于这一思想设计的。

具体规定如下：

- 若某结点无左子树，则将 `lchild` 指向其在指定遍历序列中的前驱结点。
- 若某结点无右子树，则将 `rchild` 指向其在指定遍历序列中的后继结点。

为区分指针域指向的是孩子还是线索，须在结点结构中增设两个标志域，标志域 `ltag` 和 `rtag` 的含义如下：

- `ltag` 的值为 0 表示 `lchild` 域指向左孩子，为 1 表示 `lchild` 域指向前驱。
- `rtag` 的值为 0 表示 `rchild` 域指向右孩子，为 1 表示 `rchild` 域指向后继。

线索二叉树的存储结构可定义为：

Code 5.3.1

```
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;
} ThreadNode, *ThreadTree;
```

中序线索二叉树的构造

二叉树的线索化是指将二叉链表中的空指针域替换为指向前驱或后继的线索。由于结点的前驱和后继关系只能在遍历过程中确定，因此线索化本质上就是对二叉树进行一次遍历。

通过中序遍历对二叉树线索化的递归算法如下：

Algorithm 38: InThread(p, pre)

Input: 二叉树结点 p，前驱指针 pre
Output: 线索化后的二叉树

```
begin
    if p ≠ NULL then
        InThread(p->lchild, pre)
        if p->lchild == NULL then
            p->lchild ⟵ pre
            p->ltag ⟵ 1
        if pre ≠ NULL and pre->rchild == NULL then
            pre->rchild ⟵ p
            pre->rtag ⟵ 1
        pre ⟵ p
        InThread(p->rchild, pre)
```

建立中序线索二叉树的主过程如下：

Algorithm 39: CreateInThread(T)

Input: 二叉树 T
Output: 中序线索化后的二叉树

```
begin
    pre ⟵ NULL
    if T ≠ NULL then
        InThread(T, pre)
        pre->rchild ⟵ NULL
        pre->rtag ⟵ 1
```

中序线索二叉树的遍历

中序线索二叉树的结点中已隐含前驱和后继信息，因此遍历时无须借助栈或递归。只需先找到中序序列的第一个结点，然后依次查找每个结点的后继，直至后继为空（或回到头结点）。

不含头结点的中序线索二叉树遍历相关算法如下：

1）求中序序列中的第一个结点：

Algorithm 40: FIRSTNODE(p)

```
Input: 线索二叉树结点 p
Output: 中序遍历的第一个结点
begin
    while p->ltag == 0 do
        p ← p->lchild
    return p
```

2）求结点 p 在中序序列中的后继：

Algorithm 41: NEXTNODE(p)

```
Input: 线索二叉树结点 p
Output: 中序遍历中 p 的后继结点
begin
    if p->rtag == 0 then
        return Firstnode(p->rchild)
    else
        return p->rchild
```

3）利用上面两个算法，可实现非递归的中序遍历：

Algorithm 42: INORDER(T)

```
Input: 中序线索二叉树 T
Output: 中序遍历序列
begin
    for p = Firstnode(T); p ≠ NULL; p = Nextnode(p) do
        visit(p)
```

Question 5.2

若 p、q 和 v 均为二叉树 T 中的结点，v 有两个孩子结点，T 的中序遍历序列形如“$\cdots, p, v, q, \cdots$”，则在下列叙述中，正确的是 ()。

A. p 没有右孩子，q 没有左孩子

B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子

D. p 有右孩子，q 有左孩子

A

根据中序遍历的特点，v 有左右子树，以 v 为根的子树的中序序列为：v 的左子树的中序序列，v，v 的右子树的中序序列。又因为 T 的中序序列为“$\cdots, p, v, q, \cdots$”，可知 p 属于 v 的左子树，q 属于 v 的右子树。在 v 的左子树的中序序列中：假设 p 有右孩子，则 p 的右孩子在中序序列中应在 p 之后，与 p 是最后一个遍历结点矛盾，因此 p 不存在右孩子。在 v 的右子树的中序序列中：假设 q 存在左孩子，则 q 的左孩子在中序序列中应在 q 之前，与 q 是第一个遍历结点矛盾，因此 q 不存在左孩子。

Question 5.3

设计一个算法将二叉树的叶结点按从左到右的顺序连成一个单链表，表头指针为 head。二叉树按二叉链表方式存储，链接时用叶结点的右指针域来存放单链表指针。

1）算法的基本设计思想：

通常使用的先序、中序和后序遍历对于叶结点的访问顺序都是从左到右，这里选择中序遍历。算法思想：设置前驱结点指针 pre，初始为空。第一个叶结点由指针 head 指向，遍历到叶结点时，就将它前驱的 rchild 指针指向它，最后一个叶结点的 rchild 为空。

2）算法实现：

Algorithm 43: InOrder(bt)

```
Input: 二叉树 bt
Output: 叶结点单链表头指针 head
begin
    if bt ≠ NULL then
        InOrder(bt->lchild)
        if bt->lchild == NULL and bt->rchild == NULL then
            if pre == NULL then
                head ⟵ bt
                pre ⟵ bt
            else
                pre->rchild ⟵ bt
                pre ⟵ bt
        InOrder(bt->rchild)
        pre->rchild ⟵ NULL
    return head
```

3）算法的时间复杂度为 $O(n)$，辅助变量使用 head 和 pre，栈空间复杂度为 $O(n)$。

5.4 树、森林

5.4.1 树的存储结构

双亲表示法

双亲表示法的存储结构定义如下：

Code 5.4.1

```c
#define MAX_TREE_SIZE 100
typedef struct {
    ElemType data;
    int parent;
} PTNode;
typedef struct {
    PTNode nodes[MAX_TREE_SIZE];
    int n;
} PTree;
```

双亲表示法利用了每个结点（除根节点外）有且仅有一个双亲的性质。因此，查找任意结点的双亲非常高效；然而，查找某结点的孩子则需遍历整个数组，效率较低。

孩子表示法

孩子表示法的存储结构定义如下：

Code 5.4.2

```
#define MAX_TREE_SIZE 100
struct CTNode {
    int child;
    struct CTNode *next;
};
typedef struct {
    ElemType data;
    struct CTNode *firstChild;
} CTBox;
typedef struct {
    CTBox nodes[MAX_TREE_SIZE];
    int n, r;
} CTree;
```

与双亲表示法相反，孩子表示法便于查找结点的孩子，但若要查找某结点的双亲，则需遍历所有孩子链表，检查是否包含该结点，效率较低。

孩子兄弟表示法

孩子兄弟表示法的存储结构描述如下：

Code 5.4.3

```
typedef struct CSNode {
    ElemType data;
    struct CSNode *firstChild, *nextSibling;
} CSNode, *CSTree;
```

孩子兄弟表示法具有良好的灵活性，其最大优点是能自然地将树转换为二叉树，从而复用二叉树的算法。查找孩子非常高效。然而，从当前结点高效回溯到其双亲较为困难。若在结点中增设一个 parent 指针域，则可高效查询双亲，形成一种三叉链表的存储结构。

5.4.2 树、森林与二叉树的转换

树转换为二叉树

规则：每个结点的左指针指向其第一个孩子；右指针指向其在原树中的下一个右兄弟，由于根节点没有兄弟，因此由树转换而来的二叉树的根节点必无右子树。

森林转换为二叉树

规则：首先将森林中的每棵树分别转换为对应的二叉树；由于每棵转换后的二叉树根节点均无右子树，可将这些根节点视为兄弟；将第二棵二叉树作为第一棵二叉树的右子树...以此类推，最终形成一棵完整的二叉树。

二叉树转换为森林

规则：二叉树的根及其左子树对应第一棵树的二叉树表示；将根的右指针断开，其右子树即代表剩余森林转换后的二叉树；对右子树递归应用相同对则，不断分离出下一棵树，直至右子树为空；最后将每棵树分离出的二叉树还原为普通树，即可得到原始森林。

5.4.3 树和森林的遍历

树的遍历

1）先根遍历。若树非空，则按如下规则遍历：

- 先访问根结点。
- 再依次遍历根结点的每棵子树，遍历子树时仍遵循先根后子树的规则。

2）后根遍历。若树非空，则按如下规则遍历：

- 先依次遍历根结点的每棵子树，遍历子树时仍遵循先子树后根的规则。
- 再访问根结点。

森林的遍历

1）先序遍历森林。若森林非空，则按如下规则遍历：

- 访问森林中第一棵树的根结点。
- 先序遍历第一棵树中根结点的子树森林。
- 先序遍历除去第一棵树后的剩余森林。

2）中序遍历森林，若森林非空，则按如下规则遍历：

- 中序遍历第一棵树中根结点的子树森林。
- 访问森林中第一棵树的根结点。
- 中序遍历除去第一棵树后的剩余森林。

Question 5.4

以孩子兄弟链表为存储结构，请设计递归算法求树的深度。

1）算法的基本设计思想：

采用递归算法，若树为空，高度为零；否则，高度为第一个孩子树高度加 1 和兄弟子树高度的大者。

2）算法实现：

Algorithm 44: HEIGHT(bt)

```
Input: 孩子兄弟链表表示的树 bt
Output: 树的高度
begin
    if bt == NULL then
        return 0
    else
        hc ⟵ Height(bt−>firstchild)
        hs ⟵ Height(bt−>nextsibling)
        if hc + 1 > hs then
            return hc + 1
        else
            return hs
```

5.5 树与二叉树的应用

5.5.1 哈夫曼树和哈夫曼编码

哈夫曼树的定义

Definition 5.5.1: 哈夫曼树及其相关概念

在许多实际应用中，树中的结点常被赋予一个具有特定意义的设置，称为该结点的**权**。

从树中的根节点到某一个结点的路径长度与该结点权值的乘积，称为该结点的**带权路径长度**。

树中所有叶结点的带权路径长度之和称为该树的**带权路径长度（WPL）**，记为

$$WPL = \sum_{i=1}^{n} w_i l_i$$

式中，w_i 为第 i 个叶结点的权值，l_i 为该叶结点到根结点的路径长度。

在含有 n 个带权叶节点的二叉树中，**WPL** 最小的二叉树称为**哈夫曼树**，也称最优二叉树。

哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \ldots, w_n$ 的结点，构造哈夫曼树的步骤如下：

- 将这 n 个结点分别视为 n 棵仅含一个结点的二叉树，构成森林 F。
- 构造一个新结点，从 F 中 **选取两棵根结点权值最小的树**作为其左右子树，并将新结点的权值设为左右子树根结点的权值之和。
- 从 F 中删除所选的两棵树，并将新生成的树加入 F。
- 重复步骤 2 和步骤 3，直到 F 中仅剩一棵树为止，最终得到的树即为哈夫曼树。

由上述构造过程可归纳出哈夫曼树的以下性质：

- 每个初始结点最终都成为叶结点，且权值越小的结点，其到根的路径长度越大。
- 构造过程中共新建了 $n-1$ 个结点（双分支结点），因此哈夫曼树的结点总数为 $2n-1$。
- 每次合并均选取两棵树，故哈夫曼树中不存在度为 1 的结点（为严格二叉树）。

哈夫曼编码

Definition 5.5.2: 前缀编码和哈夫曼编码

前缀编码是指任何一个码字都不是另一个码字的前缀的编码方式。

哈夫曼树中每个叶结点对应一个权值，且路径长度与权值成反比，因此哈夫曼树可用于构造哈夫曼编码。

首先，将每个字符视为一个独立的叶结点，其权值为其出现的频度；构造对应的哈夫曼树；然后，将从根到各叶结点路径上分支标记的字符串，作为该字符的编码。如图 5.1 展示了已给由哈夫曼树构造哈夫曼编码的示例，图中矩形结点标注了字符及其出现频次。

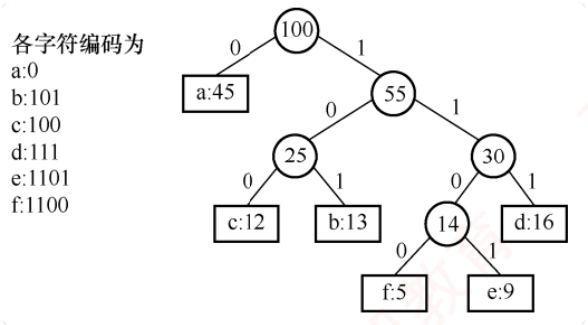

图 5.1: 由哈夫曼树构造哈夫曼编码

该哈夫曼树的 WPL 为

$$WPL = 1 \times 45 + 3 \times (13 + 12 + 16) + 4 \times (5 + 9) = 224$$

此处的 WPL 表示对 100 个字符编码后的二进制总长度，若采用 3 位固定长度编码，则总长度为 $3 \times 100 = 300$。因此，哈夫曼编码实现了约 25% 的压缩率。由此可见，利用哈夫曼树可以设计出总长度最短的二进制前缀编码。

Note:-

左分支和右分支分别表示 0 还是 1 并无强制规定，因此构造出的哈夫曼树在形态上可能不唯一。此外，当存在多个权值相同的结点时，合并顺序的不同也可能导致结构不同的哈夫曼树。

5.5.2 并查集

并查集的概念

Definition 5.5.3: 并查集

并查集是一种处理不想交集合的合并与查询问题的树状数据结构，它支持三种基本操作：

- **Initial (S)**：将集合 S 中的每个元素都初始化为一个独立的单元素子集合。
- **FindSet (S, Root1, Root2)**：将集合 S 中的子集合 Root2 并入子集合 Root1。要求 Root1 和 Root2 属于不同的子集合，否则不执行合并操作。
- **Union (S, x)**：在集合 S 中查找单元素 x 所属的子集合，并返回该子集合的根结点。

并查集的存储结构

并查集通常采用树的双亲表示法作为其存储结构：每个子集合用一棵树来表示，所有子集合对应的树共同构成一个森林，并统一存储在一个数组中。在该数组中，**数组下标代表集合中的元素编号**，而每个元素所存的值为其双亲结点的下标；对于根结点，其双亲域设为负数——通常取该子集合元素数量的相反数，既标识其为根，又记录了集合的规模。

要实现两个子集合的并操作（$S_1 \bigcup S_2$），只需将其中一个子集合根结点在数组中的值设为另一个子集合的根结点下标。

并查集的基本实现

并查集的结构定义如下：

Code 5.5.1

```
#define SIZE 100
int UFSets[SIZE];
```

（1）并查集的初始化操作

Algorithm 45: INITIAL(S)

Input: 数组 S
Output: 初始化后的并查集数组 S

```
begin
    for i = 0; i < SIZE; i++ do
        S[i] ⟵ −1
```

（2）并查集的 Find 操作

Algorithm 46: FIND(S, x)

Input: 并查集数组 S，待查找元素 x
Output: x 所属集合的根结点

```
begin
    while S[x] ≥ 0 do
        x ⟵ S[x]
    return x
```

（3）并查集的 Union 操作

Algorithm 47: UNION($S, Root1, Root2$)

Input: 并查集数组 S，两个集合的根结点 $Root1$ 和 $Root2$
Output: 合并后的并查集

```
begin
    if Root1 == Root2 then
        return
    S[Root2] ⟵ Root1
```

并查集实现的优化

在极端情况下，由 n 个元素构成的集合树可能退化成一条链，深度打到 n，此时 Find 操作的最坏时间复杂度为 $O(n)$。改进的办法是：在执行 Union 操作前先比较两个子集合的规模，将规模较小的树作为子树挂

到规模较大的树的根下，即“小树并入大树”。

（1）改进的 Union 操作

Algorithm 48: UNION($S, Root1, Root2$)

```
Input: 并查集数组 S，两个集合的根结点 Root1 和 Root2
Output: 合并后的并查集
begin
    if Root1 == Root2 then
        return
    if S[Root2] > S[Root1] then
        S[Root1] ⟵ S[Root1] + S[Root2]
        S[Root2] ⟵ Root1
    else
        S[Root2] ⟵ S[Root2] + S[Root1]
        S[Root1] ⟵ Root2
```

采用这种按集合大小合并的策略后，所构造的集合树深度不超过 $\lfloor \log_2 n \rfloor + 1$。

然而，随着子集不断合并，集合树的深度仍可能逐渐增大。为进一步缩短查找时间，还可对上述 Find 操作进一步优化：引入路径压缩机制——在找到根结点后，将从 x 到根路径上的所有中间结点直接挂接到根结点下。

（2）改进的 Find 操作

Algorithm 49: FIND(S, x)

```
Input: 并查集数组 S，待查找元素 x
Output: x 所属集合的根结点
begin
    root ⟵ x
    while S[root] ≥ 0 do
        root ⟵ S[root]
    while x ≠ root do
        t ⟵ S[x]
        S[x] ⟵ root
        x ⟵ t
    return root
```

采用按大小合并与路径压缩两种优化后，并查集的操作效率得到极大提升，使得集合树的深度不超过 $O(\alpha(n))$，其中 $\alpha(n)$ 是反 Ackermann 函数，增长极其缓慢，在实际应用中可视为常数。

Question 5.5

若任意一个字符的编码都不是其他字符编码的前缀，则称这种编码具有前缀特性。现有某字符集（字符个数 ≥ 2）的不等长编码，每个字符的编码均为二进制的 0、1 序列，最长为 L 位，且具有前缀特性。请回答下列问题：

- 哪种数据结构适宜保存上述具有前缀特性的不等长编码？
- 基于你所设计的数据结构，简述从 0/1 串到字符串的译码过程。
- 简述判定某字符集的不等长编码是否具有前缀特性的过程。

- 使用一颗二叉树保存字符集中各字符的编码，每个编码对应于从根开始到达某叶结点的一条路径，路径长度等于编码位数，路径到达的叶结点中保存该编码对应的字符。
- 从左至右依次扫描 0/1 串中的各位。从根开始，根据串中当前位置当前结点的左子指针或右子指针下移，直到移动到叶结点时为止。输出叶结点中保存的字符。然后从根开始重复这个过程，直到扫描到

0/1 串结束，译码完成。

- 二叉树既可用于保存各字符的编码，又可用于检测编码是否具有前缀特性。判定编码是否具有前缀特性的过程，也是构建二叉树的过程。初始时，二叉树中仅含有根结点，其左子指针和右子指针均为空。依次读入每个编码 C，建立/寻找从根开始对应于该编码的一条路径，过程如下：
  对每个编码，从左至右扫描 C 的各位，根据 C 的当前位置（0 或 1）沿结点的指针（左子指针或右子指针）向下移动。当遇到空指针时，创建新结点，让空指针指向该新结点并继续移动。沿指针移动的过程中，可能遇到三种情况：
  - 若遇到了叶结点（非根），则表明不具有前缀特性，返回。
  - 若在处理 C 的所有位的过程中，均没有创建新结点，则表明不具有前缀特性，返回。
  - 若在处理 C 的最后一个编码位时创建了新结点，则继续验证下一个编码。

  若所有编码均通过验证，则编码具有前缀特性。

Chapter 6

图

6.1 图的基本概念

6.1.1 图的定义

Definition 6.1.1: 图

图 G 由顶点集 V 和边集 E 组成，记为 $G=(V,E)$。其中，$V(G)$ 表示图 G 中顶点的有限非空集；$E(G)$ 表示图 G 中顶点之间的关系（边）集合。若 $V=\{v_1,v_2,\cdots,v_n\}$，则用 $|V|$ 表示图 G 中顶点的数量，$E=\{(u,v)\mid u\in V,v\in V\}$，用 $|E|$ 表示图 G 中边的数量。

Note:-

线性表可以是空表，树可以是空树，但图不可以是空图。图中不能一个顶点也没有，但边集 E 可以为空，此时图中只有顶点没有边。

6.2 图的存储及基本操作

6.2.1 邻接矩阵法

顶点数为 n 的图 $G=(V,E)$ 的邻接矩阵 A 是 $n\times n$ 的，将图 G 的顶点编号为 $v_1,v_2,\cdots,v_n$，则

$$A[i][j]=\begin{cases}1, & (v_i,v_j)\text{ 或}\langle v_i,v_j\rangle\text{ 是}E(G)\text{ 中的边}\\ 0, & (v_i,v_j)\text{ 或}\langle v_i,v_j\rangle\text{ 不是}E(G)\text{ 中的边}\end{cases}$$

对带权图而言，若顶点 v_i 和 v_j 之间有边相连，则邻接矩阵中对应位置存放该边的权值；若顶点 v_i 和 v_j 不相连，则通常用 0 或 ∞ 表示这两个顶点之间不存在边：

$$A[i][j]=\begin{cases}w_{ij}, & (v_i,v_j)\text{ 或}\langle v_i,v_j\rangle\text{ 是}E(G)\text{ 中的边}\\ 0\text{ 或}\infty, & (v_i,v_j)\text{ 或}\langle v_i,v_j\rangle\text{ 不是}E(G)\text{ 中的边}\end{cases}$$

图的邻接矩阵存储结构定义如下：

Code 6.2.1

```
#define MaxVertexNum 100
typedef char VertexType;
typedef int EdgeType;
typedef struct {
    VertexType vex[MaxVertexNum];
    EdgeType edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;
```

6.2.2 邻接表法

邻接表，是指对图 G 中的每个顶点 v_i 建立一个单链表。第 i 个单链表中的结点表示依附于顶点 v_i 的边（对于有向图，这些边是以 v_i 为尾的弧），该链表称为顶点 v_i 的边表（有向图中也称出边表）。边表的头指针与顶点的数据信息采用顺序存储，称为顶点表。因此，邻接表中包含两类结点：顶点表结点和边表结点，其结构如下所示。

顶点域	边表头指针
data	firstarc

顶点表结点

邻接点域	指针域
adjvex	nextarc

边表结点

顶点表结点由两个域组成：顶点域（data）存储顶点 v_i 的相关信息，边表头指针域（firstarc）指向其第一条邻接边的边表结点。边表结点至少包含两个域：邻接点域（adjvex）存储与该顶点邻接的另一顶点编号，指针域（nextarc）指向下一条边的边表结点。

图的邻接表存储结构定义如下：

Code 6.2.2

```
#define MaxVertexNum 100
typedef struct ArcNode {
    int adjvex;
    struct ArcNode *nextarc;
} ArcNode;
typedef struct VNode {
    VertexType data;
    ArcNode *firstarc;
} VNode, AdjList[MaxVertexNum];
typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

6.2.3 十字链表法

十字链表是有向图的一种链式存储结构。在十字链表中，每条弧用一个结点（称为弧结点）表示，每个顶点也用一个结点（称为顶点结点）表示。这两种结点的结构如下所示。

弧结点				
tailvex	headvex	hlink	tlink	(info)

顶点结点		
data	firstin	firstout

弧结点包含五个域：tailvex 和 headvex 域分别存放弧尾顶点和弧头顶点的编号；头链域 hlink 指向弧头相同的下一条弧；尾链域 tlink 指向弧尾相同的下一条弧；info 域存放该弧的相关信息。这样，弧头相同的弧在同一个链表上，弧尾相同的弧也在同一个链表上。

顶点结点包含三个域：data 域存放该顶点的数据信息，如顶点名称；firstin 域指向以该顶点为弧头的第一条弧；firstout 域指向以该顶点为弧尾的第一条弧。

6.2.4 邻接多重表法

邻接多重表是无向图的一种链式存储结构。虽然邻接表可以方便地获取顶点和边的信息，但在执行如判断两个顶点之间是否存在边等操作时，需分别遍历两个顶点的边表，效率较低。邻接多重表通过将每条边仅用一个结点表示，从而在判断边时只需遍历一次，提升了效率。

与十字链表类似，在邻接多重表中，每条边用一个结点表示，其结构如下。

ivex	ilink	jvex	jlink	(info)

其中，ivex 和 jvex 域存放该边依附的两个顶点的编号；ilink 域指向依附于顶点 ivex 的下一条边；jlink 域指向依附于顶点 jvex 的下一条边，info 域存放该边的相关信息。

每个顶点也用一个结点表示，包含如下所示的两个域。

data	firstedge

其中，data 域存放该顶点的相关信息，firstedge 域指向依附于该顶点的第一条边。

在邻接多重表中，所有依附于同一顶点的边串联在同一链表中。由于每条边依附于两个顶点，其对应的边结点同时链接在两个链表中。对无向图而言，邻接多重表仅用一个结点表示一条无向边。

表 6.1: 图的四种存储方式的总结

	邻接矩阵	邻接表	十字链表	邻接多重表
空间复杂度	$O(\|V\|^2)$	无向图：$O(\|V\|+2\|E\|)$ 有向图：$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
找相邻边	遍历对应行或列的时间复杂度为 $O(\|V\|)$	找有向图的入度必须遍历整个邻接表	很方便	很方便
删除边或顶点	删除边很方便，删除顶点需要大量移动数据	无向图中删除边或顶点都不方便	很方便	很方便
适用于	稠密图	稀疏图和其他	只能存有向图	只能存无向图
表示方式	唯一	不唯一	不唯一	不唯一

6.2.5 图的基本操作

图的基本操作独立于其存储结构，但不同的存储方式下的具体实现算法性能会有所不同。在设计具体算法时，需根据图的存储结构选择合适的数据结构和算法，以达到最佳的时间和空间效率。

图的基本操作主要包括（仅抽象地考虑，所以忽略各变量的类型）：

- **Adjacent (G, x, y)**：判断图 G 中是否存在边 $\langle x, y\rangle$ 或 (x, y)。
- **Neighbors (G, x)**：列出图 G 中与顶点 x 邻接的所有边。
- **InsertVertex (G, x)**：在图 G 中插入顶点 x。
- **DeleteVertex (G, x)**：从图 G 中删除顶点 x 及其相关的所有边。
- **AddEdge (G, x, y)**：若无向边 (x, y) 或有向边 $\langle x, y\rangle$ 不存在，则将其添加到图 G 中。
- **RemoveEdge (G, x, y)**：若无向边 (x, y) 或有向边 $\langle x, y\rangle$ 存在，则从图 G 中删除该边。
- **FirstNeighbor (G, x)**：查找图 G 中顶点 x 的第一个邻接点。若存在则返回顶点号；若 x 没有邻接点或图中不存在 x，则返回 -1。
- **NextNeighbor (G, x, y)**：假设顶点 y 是顶点 x 的一个邻接点，返回除 y 外顶点 x 的下一个邻接点的顶点号。若 y 是 x 的最后一个邻接点，则返回 -1。
- **Get_edge_value (G, x, y)**：获取图 G 中边 (x, y) 或 $\langle x, y\rangle$ 对应的权值。
- **Set_edge_value (G, x, y, v)**：设置图 G 中边 (x, y) 或 $\langle x, y\rangle$ 对应的权值为 v。

Note:-

在图的遍历算法中，会经常用到 **FirstNeighbor** 和 **NextNeighbor** 这两个操作来获取一个顶点的所有邻接点。对于不同的图的存储结构，这两个操作的实现方式会有所不同。例如，在邻接矩阵中，**FirstNeighbor** 可以通过扫描对应行或列来找到第一个邻接点，而在邻接表中，则直接访问边表即可。

Question 6.1

已知无向连通图 G 由顶点集 V 和边集 E 组成，$|E| > 0$，当 G 中度数为奇数的顶点个数为不大于 2 的偶数时，G 存在包含所有边且长度为 $|E|$ 的路径（称为 EL 路径）。设图 G 采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numVertices, numEdges;      // 图中的顶点数和边数
    char VerticesList[MAXV];        // 顶点表，MAXV 为已定义常量
    int Edge[MAXV][MAXV];           // 邻接矩阵
} MGraph;
```

请设计算法 `isExistEL(MGraph G)`，判断 G 是否存在 EL 路径，若存在，则返回 1，否则返回 0。要求：

- 给出算法的基本设计思想。
- 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- 说明你所设计算法的时间复杂度和空间复杂度。

1）算法的基本设计思想：

对于采用邻接矩阵存储的无向图，在邻接矩阵的每一行（列）中，非零元素的个数为本行（列）对应顶点的度。可以依次计算连通图 G 中各顶点的度，并记录度为奇数的顶点个数，若个数为 0 或 2，则返回 1，否则返回 0。

2）算法实现：

Algorithm 50: IsExistEL(G)

```
Input: 邻接矩阵存储的图 G
Output: 是否存在 EL 路径，存在返回 1，否则返回 0
begin
    count ⟵ 0
    for i = 0; i < G.numVertices; i++ do
        degree ⟵ 0
        for j = 0; j < G.numVertices; j++ do
            degree ⟵ degree + G.Edge[i][j]
        if degree%2 ≠ 0 then
            count ⟵ count + 1
    if count == 0 or count == 2 then
        return 1
    else
        return 0
```

3）算法需要遍历整个邻接矩阵，所以时间复杂度为 $O(n^2)$ 空间复杂度为 $O(1)$。

6.3 图的遍历

与树的遍历相比，图的遍历要复杂得多，访问某个结点后，沿着某条路径继续搜索时，可能再次回到该顶点。为避免重复访问，遍历过程中需记录已访问过的顶点，通常设置一个辅助数组 `visited[]` 来标记各顶点是否已被访问。

6.3.1 广度优先搜索（BFS）

Definition 6.3.1: 广度优先搜索

广度优先搜索（Breadth-First Search，BFS）类似于树的层序遍历。BFS 是一种分层遍历过程：每完成一层的访问，便会处理下一层的所有顶点，而不会沿着已访问路径回溯，因此通常以非递归方式实现。BFS 算法必须借助一个辅助队列，用于暂存当前层顶点的下一层邻接顶点。

广度优先搜索算法的实现如下：

Algorithm 51: BFSTraverse(G)

```
Input: 图 G
Output: 广度优先遍历结果
begin
    visited[MAX_VERTEX_NUM]
    for i = 0; i < G.vexnum; i++ do
        visited[i] ⟵ FALSE
    InitQueue(Q)
    for i = 0; i < G.vexnum; i++ do
        if !visited[i] then
            BFS(G, i)
```

基于邻接表的广度优先搜索实现如下：

Algorithm 52: BFS(G, i)

```
Input: 邻接表存储的图 G，起始顶点 i
Output: 广度优先遍历
begin
    visit(i)
    visited[i] ⟵ TRUE
    EnQueue(Q, i)
    while !QueueEmpty(Q) do
        DeQueue(Q, v)
        for p ⟵ G.vertices[v].firstarc; p; p ⟵ p−>nextarc do
            w ⟵ p−>adjvex
            if visited[w] == FALSE then
                visit(w)
                visited[w] ⟵ TRUE
                EnQueue(Q, w)
```

基于邻接矩阵的广度优先搜索实现如下：

Algorithm 53: BFS(G, i)

```
Input: 邻接矩阵存储的图 G，起始顶点 i
Output: 广度优先遍历
begin
    visit(i)
    visited[i] ⟵ TRUE
    EnQueue(Q, i)
    while !QueueEmpty(Q) do
        DeQueue(Q, v)
        for w ⟵ 0; w < G.vexnum; w ⟵ w + 1 do
            if visited[w] == FALSE and G.edge[v][w] == 1 then
                visit(w)
                visited[w] ⟵ TRUE
                EnQueue(Q, w)
```

BFS 算法的性能分析

无论采用邻接表还是邻接矩阵存储，BFS 算法都需要借助一个辅助队列 Q 来实现逐层访问。每个顶点最多入队一次，在最坏的情况下，空间复杂度为 $O(|V|)$。

采用邻接表存储时，每个顶点均需搜索（或入队）一次，时间复杂度为 $O(|V|)$；在搜索每个顶点的邻接点时，每条边均需访问一次，时间复杂度为 $O(|E|)$；总时间复杂度为 $O(|V|+|E|)$。采用邻接矩阵存储时，查找每个顶点的邻接点所需的时间为 $O(|V|)$，总时间复杂度为 $O(|V|^2)$。

BFS 算法求解单源最短路径问题

若图 $G=(V,E)$ 为非带权图，定义从顶点 u 到顶点 v 的最短路径 $d(u,v)$ 为所有从 u 到 v 的路径中所含边数的最小值；若从 u 到 v 不可达，则 $d(u,v)=\infty$。

利用 BFS 可求解非带权图的单源最短路径问题，这是因为广度优先搜索总是按照距离由近到远的顺序遍历图中顶点，从而保证每个顶点首次被访问时，所经过的路径即为最短路径。

BFS 求解单源最短路径问题的实现如下：

Algorithm 54: BFS_MIN_DISTANCE(G, u)

```
Input: 图 G，源点 u
Output: 数组 d，记录从 u 到各顶点的最短路径长度
begin
    for i ⟵ 0; i < G.vexnum; i ⟵ i + 1 do
        d[i] ⟵ ∞
    visited[u] ⟵ TRUE
    d[u] ⟵ 0
    EnQueue(Q, u)
    while !QueueEmpty(Q) do
        DeQueue(Q, u)
        for w ⟵ FirstNeighbor(G, u); w ≥ 0; w ⟵ NextNeighbor(G, u, w) do
            if !visited[w] then
                visited[w] ⟵ TRUE
                d[w] ⟵ d[u] + 1
                EnQueue(Q, w)
```

6.3.2 深度优先搜索（DFS）

Definition 6.3.2: 深度优先搜索

深度优先搜索（Depth-First Search，DFS）类似于树的先序遍历。DFS 是一种回溯算法：从起始顶点开始，沿着某条路径尽可能深入地访问顶点，当遇到死路时，便回溯到上一个顶点，继续探索其他未访问的邻接顶点，直到所有顶点都被访问完毕。

深度优先搜索算法的实现如下：

Algorithm 55: DFSTRAVERSE(G)

```
Input: 图 G
Output: 深度优先遍历
begin
    visited[MAX_VERTEX_NUM]
    for i ⟵ 0; i < G.vexnum; i ⟵ i + 1 do
        visited[i] ⟵ FALSE
    for i ⟵ 0; i < G.vexnum; i ⟵ i + 1 do
        if !visited[i] then
            DFS(G, i)
```

基于邻接表的深度优先搜索实现如下：

Algorithm 56: DFS(G, i)

```
Input: 邻接表存储的图 G，起始顶点 i
Output: 深度优先遍历
begin
    visit(i)
    visited[i] ⟵ TRUE
    for p ⟵ G.vertices[i].firstarc; p; p ⟵ p->nextarc do
        j ⟵ p->adjvex
        if visited[j] == FALSE then
            DFS(G, j)
```

基于邻接矩阵的深度优先搜索实现如下：

Algorithm 57: DFS(G, i)

Input: 邻接矩阵存储的图 G，起始顶点 i
Output: 深度优先遍历

```
begin
    visit(i)
    visited[i] ⟵ TRUE
    for j ⟵ 0; j < G.vexnum; j ⟵ j + 1 do
        if visited[j] == FALSE and G.edge[i][j] == 1 then
            DFS(G, j)
```

DFS 算法的性能分析

DFS 算法通常以递归形式实现，其执行过程中依赖系统提供的递归工作栈，在最坏的情况下，递归深度可达 $O(|V|)$，因此空间复杂度为 $O(|V|)$。

图的遍历本质上是通过边依次访问邻接点的过程。因此，无论是深度优先搜索还是广度优先搜索，其时间复杂度仅取决于图的存储结构。对于邻接表存储的图，时间复杂度为 $O(|V|+|E|)$；对于邻接矩阵存储的图，时间复杂度为 $O(|V|^2)$。

6.4 图的应用

6.4.1 最小生成树

Definition 6.4.1: 最小生成树

对于带权连通无向图 G 而言，不同的生成树其总权重（树中所有边的权值之和）可能不同。具有最小总权重的生成树称为图 G 的最小生成树（Minimum Spanning Tree，MST）。

Note:-

最小生成树中所有边的权值之和最小，但不能保证任意两个顶点之间的路径是最短路径。

构造最小生成树的方法有多种，但大多数算法都基于这样一个核心性质：假设 $G=(V,E)$ 是一个带权连通无向图，U 是顶点集 V 的一个非空子集。若 (u,v) 是一条权值最小的边，其中 $u\in U, v\in V-U$，则图 G 中必存在一棵包含边 (u,v) 的最小生成树。

Prim 算法

Prim 算法步骤如下：

假设 $G=\{V,E\}$ 是连通图，其最小生成树 $T=(U,E_T)$，E_T 是 T 中的边集合。

初始化：向空树 $T=(U,E_T)$ 中添加图 G 的任意一个顶点 u_0，使得 $U=\{u_0\}$，$E_T=\emptyset$。

循环（重复，直至 $U=V$）：从图 G 中选择满足条件 $\{(u,v)\mid u\in U, v\in V-U\}$ 且具有最小权值的边 (u,v)，将其加入树 T，并更新 $U=U\cup\{v\}$，$E_T=E_T\cup\{(u,v)\}$。

Prim 算法的简单实现如下：

Algorithm 58: PRIM(G, T)

Input: 连通图 $G = (V, E)$
Output: 最小生成树 T

```
begin
    T ⟵ ∅
    U ⟵ {w}
    while (V − U) ≠ ∅ do
        (u, v) ⟵ 使u ∈ U 与v ∈ (V − U)，且权值最小的边
        T ⟵ T ∪ {(u, v)}
        U ⟵ U ∪ {v}
```

在 Prim 算法中，每步都从当前已构建的树向外扩展一条最短边，逐步生长出整棵最小生成树。该算法的时间复杂度为 $O(|V|^2)$，与边数 $|E|$ 无关，因此特别适用于求解边稠密图的最小生成树。虽然采用其他方法能改进 Prim 算法的时间复杂度，但会增加实现的复杂度。

Kruskal 算法

Kruskal 算法步骤如下：

假设 $G = (V, E)$ 是连通图，其最小生成树 $T = (U, E_T)$，E_T 是 T 中的边集合。

初始化：$U = V, E_T = \emptyset$。即每个顶点构成一棵独立的树，T 此时是一个仅含 $|V|$ 个顶点的森林。

循环（重复，直至 T 成为一棵树）：按照边权值递增的顺序，从 $E - E_T$ 中选择一条边。若该边加入 T 后不构成回路，则将其加入 E_T；否则舍弃，直到 E_T 包含 $n - 1$ 条边为止。

Kruskal 算法的简单实现如下：

Algorithm 59: KRUSKAL(V, T)

Input: 图 $G = (V, E)$，顶点数 n
Output: 最小生成树 T

```
begin
    T ⟵ V
    numS ⟵ n
    while numS > 1 do
        (v, u) ⟵ 从E 中取出权值最小的边
        if v 和 u 属于 T 中不同的连通分量 then
            T ⟵ T ∪ {(v, u)}
            numS ⟵ numS − 1
```

在 Kruskal 算法中，每当选择一条连接两棵不同树的边时，这两棵树将通过这条边合并为一棵更大的树，随着算法的进行，整个森林逐渐合并成一棵树。考虑到算法效率，在最坏情况下需要对所有的 $|E|$ 条边各扫描一次。通常，边会存储在一个堆中，每次从中选出最小权值的边所需时间为 $O(\log_2 |E|)$。同时，使用并查集来快速确定两个顶点是否属于同一集合的时间复杂度为 $O(\alpha(|V|))$，这里 $\alpha(|V|)$ 增长极其缓慢，可视为常数。因此，Kruskal 算法的总时间复杂度为 $O(|E| \log_2 |E|)$，不依赖于 $|V|$，这使得它特别适合于处理边稀疏但顶点较多的图。

6.4.2 最短路径

Dijkstra 算法求单源最短路径

Dijkstra 算法使用一个集合 S 记录已确定最短路径的顶点。初始时，将源点（顶点 0）放入 S；每当将一个新顶点 i 加入 S 后，需更新源点到所有尚未确定最短路径的顶点（集合 $V - S$ 中的顶点）的当前最短路径长度。算法执行过程中维护以下三个辅助数组：

- **final[]**：标记各顶点是否已找到最短路径（是否属于集合 S）。
- **dist[]**：记录从源点到各顶点的当前最短路径长度。初始化时，若存在从源点到顶点 i 的直接边，则 $dist[i]$ 为该边的权值，否则置为 ∞。
- **path[]**：path[i] 存储从源点到顶点 i 的最短路径。算法结束后，可通过 path[] 数组回溯，重构出完整的最短路径。

假设源点为顶点 0，集合 S 初始仅含顶点 0。图以邻接矩阵 arcs 表示，其中 arcs[i][j] 为有向边 $\langle i, j\rangle$ 的权值；若该边不存在，则 arcs[i][j] 为 ∞。

Dijkstra 算法的步骤如下（暂不考虑对 path[] 的操作）：

- **1）初始化**：集合 $S = \{0\}$，dist[] 的初始值为 $dist[i] = arcs[0][i],\ i = 1, 2, \cdots, n-1$。
- **2）选择最短路径顶点**：从不在 S 中的顶点集合 $(V - S)$ 中选出顶点 j，使得 $dist[j]$ 最小，此时，顶点 j 即为当前从源点出发的最短路径的终点，将其加入集合 S。
- **3）松弛操作**：对每个从顶点 j 出发的邻接顶点 k（$arcs[j][k] \neq \infty$），若 $dist[j] + arcs[j][k] < dist[k]$，则更新 $dist[k] = dist[j] + arcs[j][k]$。
- **4）重复步骤 2） 3）共 $n-1$ 次**，直至所有顶点都包含在集合 S 中。

每个新顶点加入集合 S 后，都可能发现到达其他尚未确定最短路径顶点的更短路径，从而需要更新相应的 dist[] 值。

使用邻接矩阵表示图，并采用线性扫描 dist[] 查找最小值时，Dijkstra 算法的时间复杂度为 $O(|V|^2)$。若改用带权邻接表，虽然松弛操作的总代价可降至 $O(|E|)$，但由于查找最小 dist[] 值仍需 $O(|V|)$ 时间，总时间复杂度仍为 $O(|V|^2)$。即使只需求解从源点到某一个特定顶点的最短路径，在最坏情况下仍需处理所有顶点，因此时间复杂度不变，仍为 $O(|V|^2)$。

特别注意：Dijkstra 算法不适用于存在负权边的图。该算法基于贪心策略，一旦顶点被加入集合 S，便不再更新其最短路径。然而，若图中存在负权边，后续路径可能通过负权边“绕回”已确定的顶点，从而发现更短路径，而 Dijkstra 算法无法察觉这一变化，导致结果错误。

Floyd 算法求各顶点之间最短路径问题

给定带权有向图，对任意两个不同顶点 $v_i \neq v_j$，求解从 v_i 到 v_j 的最短路径及其长度。

Floyd 算法的基本思想：迭代生成一系列 n 阶方阵 $A^{(-1)}, A^{(0)}, \cdots, A^{(k)}, \cdots, A^{(n-1)}$，其中 $A^{(k)}[i][j]$ 表示从顶点 v_i 到 v_j、仅允许使用编号不超过 k 的顶点作为中间顶点时的最短路径长度。初始时 $(k = -1)$，若存在从 v_i 到 v_j 的直接边，则 $A^{(-1)}[i][j]$ 为该边的权值；否则设为 ∞。特别地，对所有 i，令 $A^{(-1)}[i][i] = 0$。随后，依次考虑顶点 k $(k = 0, 1, \cdots, n-1)$ 作为中间顶点，若经由 v_k 的路径比当前记录的路径更短，则更新对应的距离。算法的描述如下：

定义 n 阶方阵序列 $A^{(-1)}, A^{(0)}, \cdots, A^{(n-1)}$，其中，

$$A^{(-1)}[i][j] = \text{arcs}[i][j]$$

$$A^{(k)}[i][j] = \min\left\{A^{(k-1)}[i][j],\ A^{(k-1)}[i][k] + A^{(k-1)}[k][j]\right\}, \quad k = 0, 1, \cdots, n-1$$

式中，$A^{(0)}[i][j]$ 表示从 v_i 到 v_j，仅允许使用 v_0 作为中间顶点的最短路径长度；一般地，$A^{(k)}[i][j]$ 表示从 v_i 到 v_j，**中间顶点编号不超过 k 的最短路径长度**。Floyd 算法是一个迭代过程，每完成一次迭代，便将下一个顶点纳入可选的中间顶点集合；经过 n 次迭代后，$A^{(n-1)}[i][j]$ 即为 v_i 到 v_j 的最短路径长度。最终，矩阵 $A^{(n-1)}$ 中的每个元素即为对应顶点对的最短路径长度。

Floyd 算法的时间复杂度为 $O(|V|^3)$。其代码结构紧凑，仅需一个三重循环，无须复杂数据结构，且常数因子较小，因此在中等规模图上实际运行效率较高。

Floyd 算法允许图中存在负权边，但不允许存在负权环。Floyd 算法同样适用于带权无向图——只需将每条无向边视为两条方向相反、权值相同的有向边。

此外，也可通过调用单源最短路径算法求解所有顶点对的最短路径：在边权非负的条件下，依次以每个顶点为源点运行 Dijkstra 算法。Dijkstra 算法的时间复杂度为 $O(|V|^2)$，因此需执行 $|V|$ 次，总时间复杂度为 $O(|V|^3)$，与 Floyd 算法相同。

表 6.2: BFS 算法、Dijkstra 算法和 Floyd 算法求最短路径的总结

	BFS 算法	**Dijkstra 算法**	**Floyd 算法**
用途	求单源最短路径	求单源最短路径	求各顶点之间的最短路径
无权图	适用	适用	适用
带权图	不适用	适用	适用
带负权值的图	不适用	不适用	适用
带负权回路的图	不适用	不适用	不适用
时间复杂度	$O(\|V\|^3)$ 或 $O(\|V\|+\|E\|)$	$O(\|V\|^2)$	$O(\|V\|^3)$

6.4.3 有向无环图描述表达式

有向无环图描述表达式参考 B 站视频：有向无环图描述表达式。

6.4.4 拓扑排序

Definition 6.4.2: AOV 网

若用有向无环图表示一个工程，每个顶点代表一个活动，并用有向边 $\langle V_i, V_j\rangle$ 表示活动 V_i 必须先于活动 V_j 进行的关系，则称这种有向图为顶点表示活动的网络，简称 AOV 网。

在此网络中，活动 V_i 是活动 V_j 的直接前驱，而 V_j 则是 V_i 的直接后继，这种关系具有传递性。此外，任何活动都不能成为自己的前驱或后继。

在图论中，拓扑排序指的是由有向无环图的顶点构成的一个线性序列，该序列满足条件：

- 每个顶点恰好出现一次。
- 若存在从顶点 A 到 B 的路径，则在拓扑排序中，A 必须出现在 B 之前。

每个 AOV 网都有一个或多个拓扑排序序列。

一种常用的拓扑排序方法步骤：

- 从 AOV 网中选择一个没有前驱（入度为 0）的顶点并输出之。图中可能存在多个入度为 0 的顶点，随机选择一个输出，因此拓扑序列可能不唯一。
- 从网中删除该顶点和所有以它为起点的有向边。
- 重复步骤上述步骤直到当前 AOV 网为空或无法找到新的入度为 0 的顶点为止。后一种情况表明有向图中必然存在环。

基于邻接表存储结构的拓扑排序算法的实现如下：

Algorithm 60: TopologicalSort(G)

```
Input: 图 G（邻接表存储）
Output: 拓扑排序成功返回 true，失败返回 false
begin
    InitStack(S)
    for i = 0; i < G.vexnum; i++ do
        if indegree[i] == 0 then
            Push(S, i)
    count ⟵ 0
    while !StackEmpty(S) do
        Pop(S, i)
        print[count] ⟵ i
        count ⟵ count + 1
        p ⟵ G.vertices[i].firstarc
        for p = G.vertices[i].firstarc; p ≠ NULL; p = p->nextarc do
            v ⟵ p->adjvex
            if !(--indegree[v]) then
                Push(S, v)
    if count < G.vexnum then
        return false
    else
        return true
```

因为输出每个顶点的同时还要删除以它为起点的边，所以采用邻接表存储时，拓扑排序的时间复杂度为 $O(|V|+|E|)$；采用邻接矩阵存储时，时间复杂度为 $O(|V|^2)$。

通过深度优先搜索（DFS）也可以实现拓扑排序，其基本思路如下。对于有向无环图 G 中的任意两个顶点 u 和 v，其关系必属于以下三种情形之一：

- 若 u 是 v 的祖先（存在从 u 到 v 的路径），则在 DFS 过程中会先访问 u，再递归访问 v；v 先完成回溯，因此 v 的结束时间早于 u，即 u 的结束时间大于 v 的结束时间。
- 若 u 是 v 的子孙，则 v 是 u 的祖先，因此 v 的结束时间大于 u 的结束时间。
- 若 u 和 v 之间不存在任何方向的路径，则它们在拓扑序列中的相对顺序可以任意。

于是，将所有顶点按 DFS 结束时间降序排列，即可得到一个合法的拓扑排序序列。

对一个 AOV 网，若采用以下步骤进行排序，则称为逆拓扑排序：

- ① 从 AOV 网中选择一个没有后继（出度为 0）的顶点并输出。
- ② 从网中删除该顶点和所有以它为终点的有向边。
- ③ 重复步骤 ① 和 ②，直到当前的 AOV 网变为空为止。

6.4.5 关键路径

Chapter 7

Kruskal's Algorithm with Data Structures

Chapter 8

Red Black Tree Data Structure

Chapter 9

Maximum Flow

矩阵缩放问题最早由 Sinkhorn 提出，后来 Cormen 等人在算法教材中也有讨论 [CTL+]。

Chapter 10

Randomized Algorithm

Chapter 11

Derandomization

Chapter 12

Global Min Cut

Chapter 13

Matching

Chapter 14

Linear Programming

Chapter 15

Approximation Algorithms using LP

Chapter 16

P, NP and Reductions

Chapter 17

参考文献

[CTL+] Thomas H. Cormen, H. Thomas, Leiserson, E. Charles, Ronald L. Rivest, L. Ronald, Stein, and Clifford. *Introduction to Algorithms*. Brand: PHI Learning.